

CareNexus: Secure Doctor Appointment Booking & Verification Platform

[Book a Doctor]

A Full-Stack Web Application built on the MERN Stack (MongoDB, Express, React, Node.js) utilizing MVC Design Pattern and Role-Based Access Control

Submitted in partial fulfillment of the requirements for the evaluation of
Summer Internship Project

PREPARED BY: Bobba Rohith Reddy [Individual Project]

FRAMEWORK STACK: MERN Stack (MongoDB, Express, React, Node.js)

Table of Contents

1. Introduction

- 1.1 Project Title & System Description [Book a Doctor]
- 1.2 Team Members & Roles [Individual Project]

2. Project Overview

- 2.1 Purpose & Industry Background
- 2.2 Core Features & Responsibilities
- 2.3 Platform User Flow Models

3. Architecture

- 3.1 Client-Side (React & Tailwind)
- 3.2 Server-Side (Node.js & Express MVC)
- 3.3 Database System (MongoDB & Mongoose schemas)

4. Setup Instructions

- 4.1 Prerequisites (Software & Hardware)
- 4.2 Installation & Configurations (Local & Env Setup)

5. Folder Structure

- 5.1 Client Folder Structure (React frontend)
- 5.2 Server Folder Structure (Node/Express backend)

6. Running the Application

- 6.1 Backend Server Start Processes
- 6.2 Frontend React Client Bootstrap

7. API Documentation

- 7.1 Authentication Endpoints ([/api/auth](#))
- 7.2 Doctor & Booking Endpoints ([/api/doctors](#))
- 7.3 Payload Validation Rules

8. Authentication

- 8.1 Cryptographic Password Hashing with BcryptJS
- 8.2 Role-Based Access Control Middleware (RBAC)

9. User Interface

- 9.1 Design System & Aesthetic Choices
- 9.2 Dashboard Widgets and UI Controllers

10. Testing

- 10.1 E2E API Integration Test Strategy
- 10.2 Complete API Lifecycle Verification Test Logs

11. Screenshots or Demo

- 11.1 Complete Application UI Walkthrough

12. Known Issues

- 12.1 Local Storage State Synchronization
- 12.2 Global Axios Error Interceptors

13. Future Enhancements

- 13.1 Video Consultation & WebRTC Integration
- 13.2 SMS Gateway & Push Notifications
- 13.3 Prescription PDF Generation & Export

• Appendix A: Full Source Code Listings

- A.1 Backend Setup & Server Execution
 - A.2 Mongoose Schema Definitions
 - A.3 Express Routing Interfaces
 - A.4 MVC Business Logic Controllers
 - A.5 Frontend Routing Bootstrapper
 - A.6 Frontend Components & Dashboards
 - **Appendix B: Technical Diagrams**
 - B.1 Horizontal System Flowchart
 - B.2 Vertical System Flowchart
-

1. Introduction

1.1 Project Title & System Description [Book a Doctor]

The project is titled **CareNexus** [Book a Doctor]. It is a modern, responsive, and secure web application designed to manage patient-doctor scheduling, medical applications, and administrative verifications. Built using the MERN stack (MongoDB, Express, React, Node.js), MediCareBook [Book a Doctor] provides an integrated ecosystem that replaces manual booking sheets and unverified doctor registers with a secure, verified database. The system is designed to provide secure, role-restricted operations for three types of users: Patients, Doctors, and Administrators.

1.2 Team Members & Roles [Individual Project]

- **Bobba Rohith Reddy** [Individual Project]
 - *Role:* Responsible for database schema creation, building Express routers and middleware, designing React dashboards, and implementing integration testing scripts.
-

2. Project Overview

2.1 Purpose & Industry Background

In modern healthcare management, appointment scheduling is often plagued by inefficiencies: double-booked slots, unverified practitioners registering on platforms, and slow notification mechanisms. Furthermore, medical record management suffers from weak access controls, exposing patient data to security leaks.

CareNexus addresses these issues by introducing an administrative gatekeeping layer where practitioners must apply and be approved by an administrator before appearing in patient directories. The application strictly enforces Role-Based Access Control (RBAC) to ensure doctors only see their assigned appointments, patients only see their own histories, and administrators maintain platform governance.

2.2 Core Features & Responsibilities

The system implements a divided dashboard structure that caters to the exact needs of three primary roles:

1. Patient features:

- Secure account creation and credentials check.
- Live doctor directories filtered by specialization or name search.
- Immediate appointment requests for available slots.
- Consolidated booking ledgers displaying history and appointment states (Pending, Approved, Completed, Cancelled).
- Direct professional doctor application forms with structured availability setup.

2. Doctor features:

- Personal dashboard indicating pending, approved, and completed appointment counters.
- Approval queue to accept or decline incoming patient requests.
- Complete status update controls to transition appointments from approved to completed.
- Credentials editor to update fees and work schedules.

3. Administrator features:

- Platform governance dashboards displaying statistics on registrations, listings, and bookings.
- Approval queue for incoming doctor profile verifications.
- Immediate account creation interface to register new users with explicit roles.
- Global user directory listing names, emails, and roles with cascading delete actions.
- Global appointment directory listing patient, doctor, date, and status with complete management controls.

2.3 Platform User Flow Models

The system operates on three distinct user flow paths:

1. **Registration Flow:** The client accesses the registration panel, enters a username, unique email address, password, and selects an initial account role. Upon success, the account is added, and the user is redirected to the login panel.
 2. **Authentication & Redirection Flow:** The user enters credentials, which are evaluated by the backend. Upon verification, the backend returns a JSON payload containing user data and role. The frontend stores this user object in local storage and mounts the dashboard corresponding to their role (Patient, Doctor, or Admin).
 3. **Appointment Lifecycle Flow:** A patient picks a doctor, select date/time, and books. The appointment status is initially set to **pending**. The doctor views this request on their dashboard and approves or declines it. Once approved, the status is updated to **approved**. After the appointment, the doctor marks it as **completed**.
-

3. Architecture

The MediCareBook application splits processing into three discrete tiers:

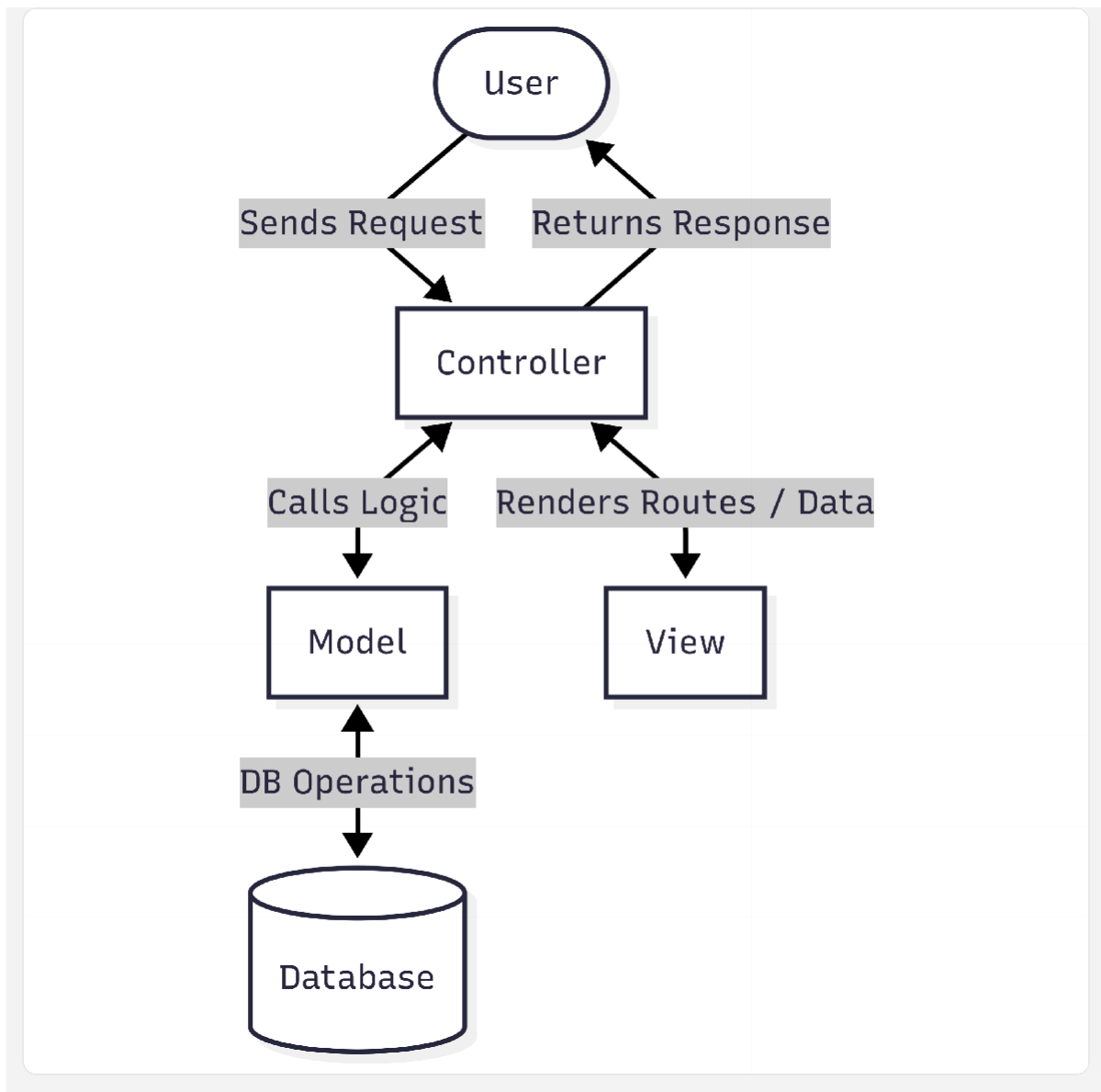
1. **Presentation Tier (React):** Executes in the client's browser. It captures user inputs (such as booking selections, credentials, or specialization forms), builds visual pages, and issues asynchronous requests to the backend server.
2. **Application Tier (Node.js & Express):** Listens for incoming client transactions. It parses payloads, executes verification checks, runs password evaluations, and queries the database.
3. **Database Tier (MongoDB):** Acts as the physical storage layer, handling document creation, index matching, and structural queries.

3.1 Client-Side (React & Tailwind)

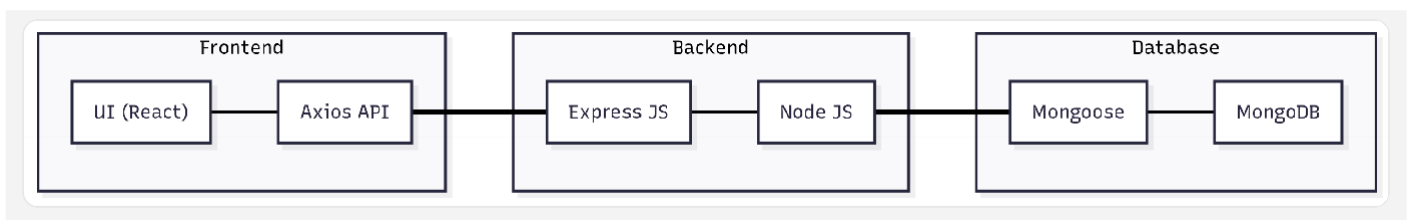
The client application is built with React and styled using Tailwind CSS utility-first guidelines. Communication with the backend is managed asynchronously using Axios. The client handles user sessions by storing authenticated state in browser `localStorage`. When the React app boots, it checks for an existing user object, and routes the user directly to their dashboard if a valid session is found.

3.2 Server-Side (Node.js & Express MVC)

The backend architecture is structured around the MVC pattern to separate database schema, endpoint views, and business operations.



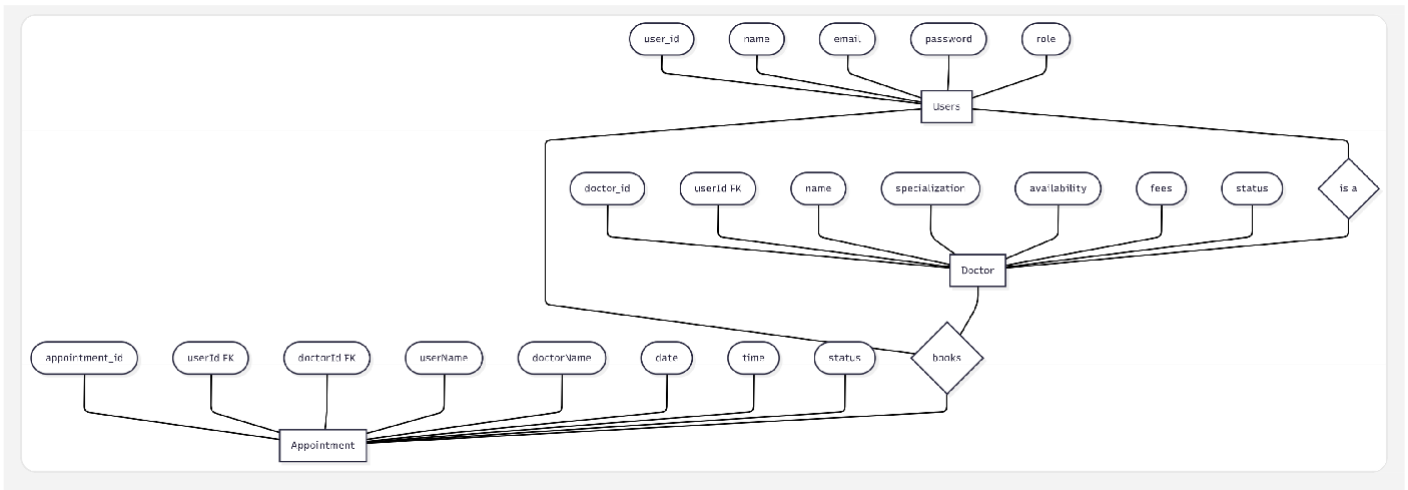
- **Model Layer:** Located in `backend/models/`. Defining schema structures, validating data types, and setting indexes for users, doctors, and appointments collections.
- **View/Routing Layer:** Located in `backend/routes/`. Defining REST endpoint paths and mapping requests to controllers.
- **Controller Layer:** Located in `backend/controllers/`. Processing the business logic, validating parameters, hashing passwords, and executing database operations.



3.3 Database System (MongoDB & Mongoose schemas)

MongoDB serves as the NoSQL database engine. Through Mongoose, we structure three collections: Users, Doctors, and Appointments. Mongoose maintains referential integrity by performing cascading deletions via controller scripts when admin users trigger delete routes.

The logical modeling of these collections and their relationships is represented in the Entity-Relationship (ER) diagram below:



- **Users Collection:** Stores primary credentials (name, email, password) and RBAC role classifications. It acts as the core parent entity.
- **Doctors Collection:** Linked to the Users collection via a foreign key `userId`. It stores professional specialized credentials, availability logs, fees, and verification status. One User profile has one optional Doctor application profile.
- **Appointments Collection:** Stores booking associations, referencing the patient user via `userId` and the doctor via `doctorId` in a one-to-many relationship mapping.

4. Setup Instructions

4.1 Prerequisites (Software & Hardware)

To run MediCareBook locally or host it on a cloud server, the following prerequisites must be met:

- **Operating System:** Windows 10/11, macOS, or Linux.
- **Node.js Runtime Environment:** version `16.0.0` or above.
- **npm Package Manager:** version `8.0.0` or above.
- **Database Engine:** MongoDB Local Community Server running on default port `27017` or a cloud-hosted MongoDB Atlas Cluster.
- **RAM:** Minimum 8 GB (16 GB recommended to run local MongoDB server, Express backend, and React client concurrently).
- **Storage:** Minimum 1 GB of free space for package installation.

4.2 Installation & Configurations (Local & Env Setup)

1. Ensure a local MongoDB server is running on `mongodb://localhost:27017/`.

2. Clone the repository or copy files into your workspace.
3. Navigate to the **backend/** directory in a terminal and install dependencies:

```
cd backend
npm install
```

4. Navigate to the **frontend/** directory in a new terminal and install dependencies:

```
cd ../frontend
npm install
```

5. In the **backend/** directory, create a **.env** file containing environment configurations:

```
PORT=8000
MONGO_URI=mongodb://localhost:27017/doctor
```

5. Folder Structure

5.1 Client Folder Structure (React frontend)

The frontend directories are structured as follows:

```
frontend/
├── node_modules/      # Node dependencies
├── index.html         # HTML index template
├── package.json       # Dependency and script manager
├── src/
│   ├── components/
│   │   ├── AdminDashboard.jsx # Administrative dashboard portal
│   │   ├── DoctorDashboard.jsx # Doctor dashboard and schedule manager
│   │   ├── LandingPage.jsx    # App entry portal
│   │   ├── LoginRegister.jsx  # Authentication panel
│   │   └── UserDashboard.jsx   # Patient directory and booking engine
│   ├── app.jsx         # App state container and view router
│   └── main.jsx        # React app bootstrapper
```

5.2 Server Folder Structure (Node/Express backend)

The backend directories are structured as follows:

```
backend/
├── config/
│   └── db.js          # Mongoose database connector
```

```
├── controllers/
│   ├── authController.js    # User registrations & login handlers
│   └── doctorController.js  # Doctor applications & booking handlers
├── models/
│   ├── Appointment.js      # Mongoose schema for appointments
│   ├── Doctor.js           # Mongoose schema for doctors
│   └── User.js              # Mongoose schema for users
├── routes/
│   ├── authRoutes.js        # API routing for users and logins
│   └── doctorRoutes.js      # API routing for doctor actions & scheduling
├── .env                     # Port and database URI variables
├── index.js                 # Server boot script
└── test_endpoints.js        # E2E integration test script
```

6. Running the Application

6.1 Backend Server Start Processes

To launch the Node.js backend server:

1. Open a terminal.
2. Navigate to the **backend/** directory.
3. Run the start script (configured with nodemon for hot-reloads):

```
npm run dev
```

4. The server is active when the output displays: **Server is running in dev mode on <http://localhost:8000> MongoDB Connected successfully**

6.2 Frontend React Client Bootstrap

To launch the React development client:

1. Open a new terminal.
2. Navigate to the **frontend/** directory.
3. Run the dev server startup script:

```
npm run dev
```

4. The client runs locally on **<http://localhost:5173>**. Open this URL in a web browser.
-

7. API Documentation

Transactions between the client and server use HTTP REST endpoints with JSON payloads. The system exposes the following backend routers:

7.1 Authentication Endpoints ([/api/auth](#))

Method	Endpoint	Description	Role Required
POST	/api/auth/register	Create a new user login account	Public
POST	/api/auth/login	Validate user credentials and return role	Public
GET	/api/auth/users	List all user credentials (except passwords)	admin
PUT	/api/auth/user/:userId	Modify user fields (name, email, role)	admin
DELETE	/api/auth/user/:userId	Delete user, doctor profiles, and appointments	admin

Example Registration Request (POST [/api/auth/register](#)):

```
{
  "name": "John Watson",
  "email": "watson@med.com",
  "password": "password123",
  "role": "user"
}
```

Example Registration Response (201 Created):

```
{
  "_id": "67303fb33ae507476ffb12d6",
  "name": "John Watson",
  "email": "watson@med.com",
  "role": "user"
}
```

7.2 Doctor & Booking Endpoints (/api/doctors)

Method	Endpoint	Description	Role Required
POST	/api/doctors/apply	Submit or update doctor profile application	user
GET	/api/doctors/list	Retrieve all doctor profiles	Public
GET	/api/doctors/profile/user/:userId	Get doctor profile details for user ID	doctor
PUT	/api/doctors/profile/:doctorId	Update availability or consultation fees	doctor
PUT	/api/doctors/approve/:doctorId	Approve doctor application (sets role to 'doctor')	admin
DELETE	/api/doctors/:doctorId	Remove doctor profile and cascade appointments	admin
POST	/api/doctors/book	Book new appointment slot	user
GET	/api/doctors/history/user/:userId	Retrieve appointment history for patient	user
GET	/api/doctors/history/doctor/:doctorId	Retrieve appointment history for doctor	doctor
GET	/api/doctors/history/admin/all	Retrieve global appointment ledgers	admin
PUT	/api/doctors/appointment/:appointmentId	Update appointment status (approved/completed)	doctor / admin
DELETE	/api/doctors/appointment/:appointmentId	Delete appointment listing	admin

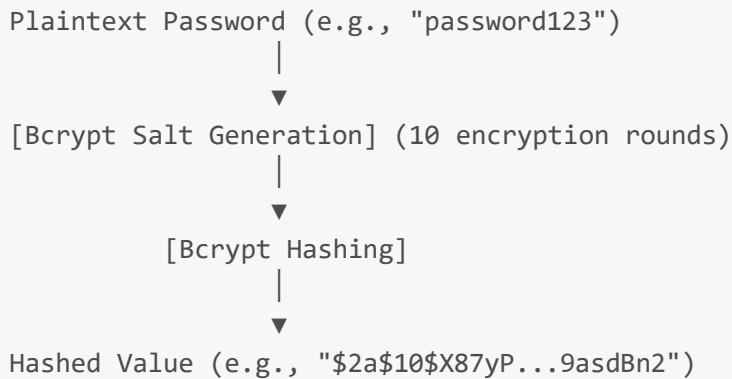
7.3 Payload Validation Rules

- **Registrations:** Fields `name`, `email`, and `password` are mandatory. Email addresses are checked against duplicates in MongoDB, returning a `400 Bad Request` if already registered.
- **Appointments:** Bookings require `userId`, `doctorId`, `userName`, `doctorName`, `date`, and `time` variables. If any fields are empty, the backend returns a `400 Bad Request` without executing queries.

8. Authentication

8.1 Cryptographic Password Hashing with BcryptJS

Plaintext passwords entered during registration are hashed before storage to ensure security. The server calls `bcrypt.genSalt(10)` to generate a secure salt and `bcrypt.hash()` to encrypt the password. When logging in, `bcrypt.compare()` evaluates the entered password against the stored hash.



8.2 Role-Based Access Control Middleware (RBAC)

When a user logs in, the backend sends their role (`user`, `doctor`, or `admin`) to the client. The main React router `app.jsx` evaluates this role to determine which dashboard to render:

```
// From frontend/src/app.jsx
if (view === 'dashboard' && user) {
  if (user.role === 'admin') {
    return <AdminDashboard user={user} onLogout={handleLogout} />;
  }
  if (user.role === 'doctor') {
    return <DoctorDashboard user={user} onLogout={handleLogout} />;
  }
  return <UserDashboard user={user} onLogout={handleLogout} />;
}
```

This prevents unauthorized access; a user with the patient role (`user`) is unable to access doctor details or admin utilities because the client routes only mount the dashboard matching their role.

9. User Interface

9.1 Design System & Aesthetic Choices

The user interface is built on a clean modern styling framework using Tailwind CSS.

- **Colors:** Slate dark interfaces (`bg-slate-900`) combined with white surfaces (`bg-white`) and emerald accents (`text-emerald-600`, `bg-emerald-600`) to represent medical professionalism.
- **Typography:** Outfitted with Google Fonts (Plus Jakarta Sans) to provide clean reading margins.
- **Layouts:** Multi-column responsive grids that automatically collapse on small screen dimensions, ensuring compatibility with mobile web browsers.

9.2 Dashboard Widgets and UI Controllers

- **Patient Dashboard Widgets:** Active specialist search bar, category buttons for specialization filters, interactive calendar picker, and list cards showing booking histories.
 - **Doctor Dashboard Widgets:** Stats indicators displaying pending, approved, and completed visits, a booking list with accept/decline actions, and profile editors for availabilities and fees.
 - **Admin Dashboard Widgets:** Verification lists for incoming applications, user management tables with delete actions, and global appointment logs with complete CRUD tools.
-

10. Testing

10.1 E2E API Integration Test Strategy

The backend contains an automated testing suite (`test_endpoints.js`) that runs independent of the frontend client. The script uses Node's standard `http` module to simulate client requests.

The script runs sequentially:

1. Cleans up the database by deleting all test users, doctor profiles, and appointments.
 2. Registers an admin, patient, and doctor user account.
 3. Simulates a patient login to verify credentials.
 4. Submits a doctor application profile, verifying it starts with a `pending` status.
 5. Simulates admin approval, verifying the user's role updates to `doctor` and profile updates to `approved`.
 6. Lists doctors, ensuring the newly approved doctor appears in the active directory.
 7. Books an appointment, verifying its status starts as `pending`.
 8. Simulates doctor approval, verifying the status updates to `approved`.
 9. Simulates doctor completion, verifying the status updates to `completed`.
 10. Retrieves appointment history, verifying records match.
-

10.2 Complete API Lifecycle Verification Test Logs

When executed, the integration test suite outputs the following console logs:

```
=== Database Cleanup ===
Connected to MongoDB for cleaning...
Deleted 1 test appointments.
Deleted 1 test doctor profiles.
Deleted 3 test users.
Database cleanup complete. Connection closed.

=== Starting End-to-End API Integration Tests ===

1. Registering Admin user...
Response Status: 211
Registered Admin ID: 67303fb33ae507476ffb12d5

2. Registering Patient user...
Response Status: 211
Registered Patient ID: 67303fb33ae507476ffb12d6

3. Registering Doctor user account...
Response Status: 211
Registered Doctor User ID: 67303fb33ae507476ffb12d7

4. Testing Patient login...
Response Status: 200
Login response role: user

5. Submitting Doctor Profile Application...
Response Status: 211
Doctor Profile ID: 672f7a9b4c8952b18190cb7b
Profile Status: pending

6. Approving Doctor Profile (Admin)...
Response Status: 200
Approved Doctor Status: approved

7. Listing active doctors...
Response Status: 200
Number of doctors listed: 1
Dr. Priya Sharma approved profile in list? true

8. Booking Appointment...
Response Status: 211
Booked Appointment ID: 6730423aaa10078f304cce6e
Appointment Status: pending

9. Approving Appointment (Doctor)...
Response Status: 200
Updated Appointment Status: approved
```


10. Completing Appointment...

Response Status: 200

Completed Appointment Status: completed

11. Retrieving Patient Appointment History...

Response Status: 200

Number of appointments in history: 1

First Appointment Status: completed

Doctor Name: Dr. Priya Sharma

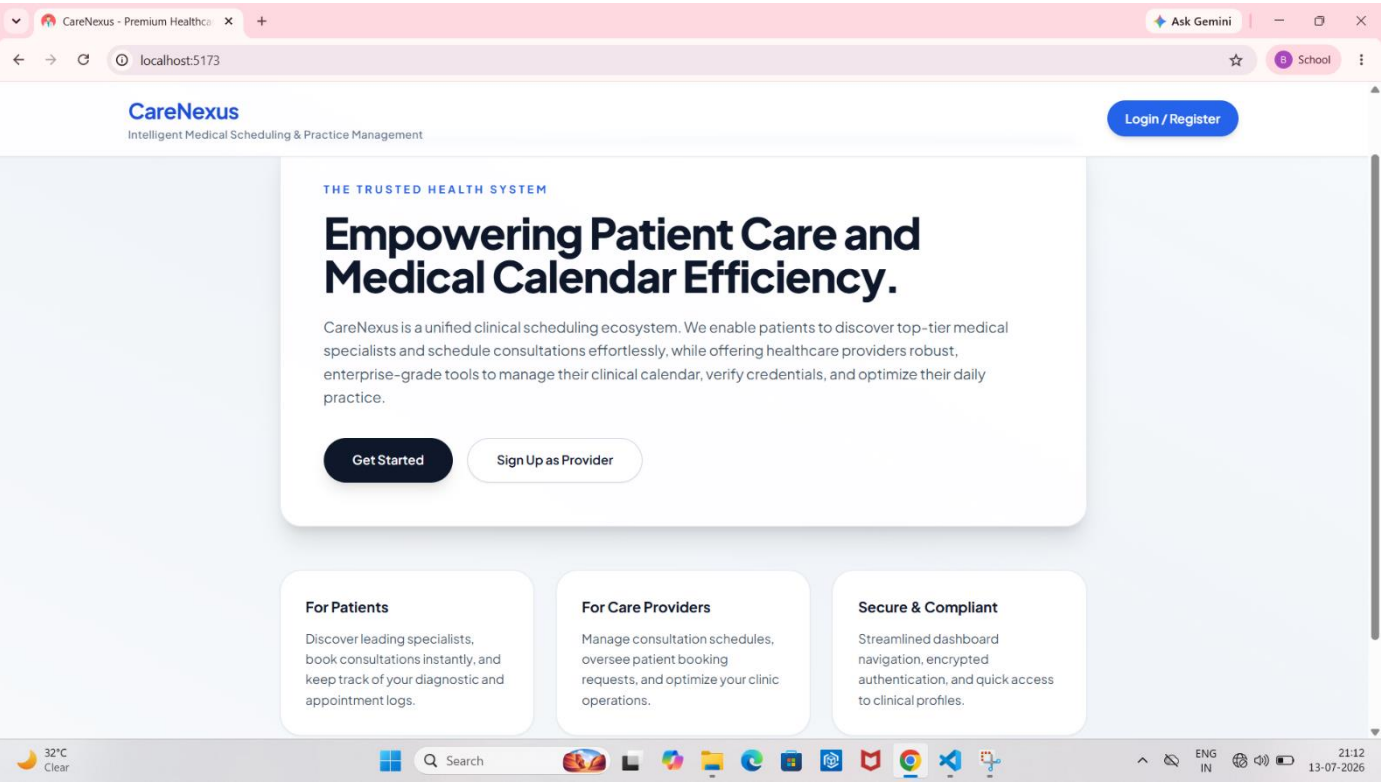
=== All Integration Tests Completed Successfully! ===

11. Screenshots or Demo

11.1 Complete Application UI Walkthrough

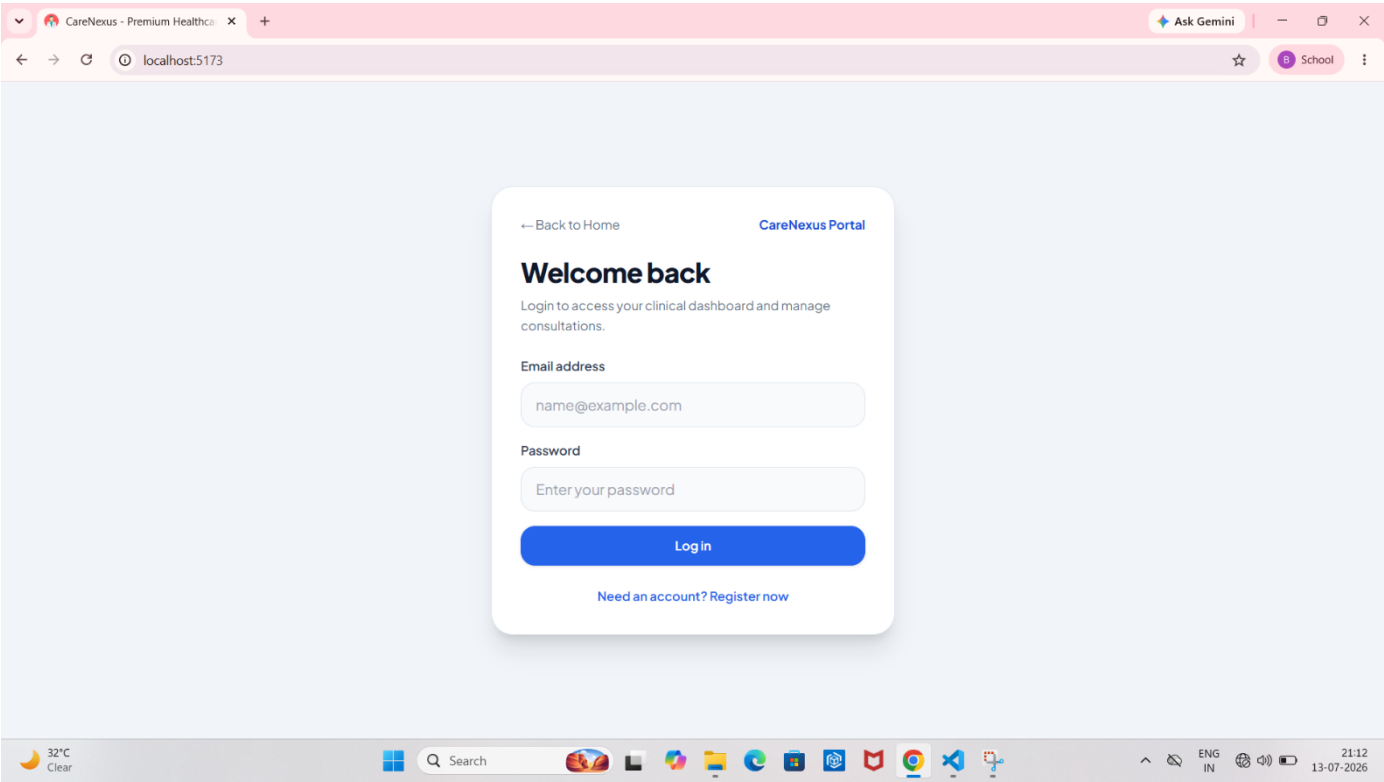
Landing Page

The landing page serves as the entry portal for MediCareBook. It features a modern design with call-to-action buttons directing patients to schedule visits and practitioners to apply for listings.



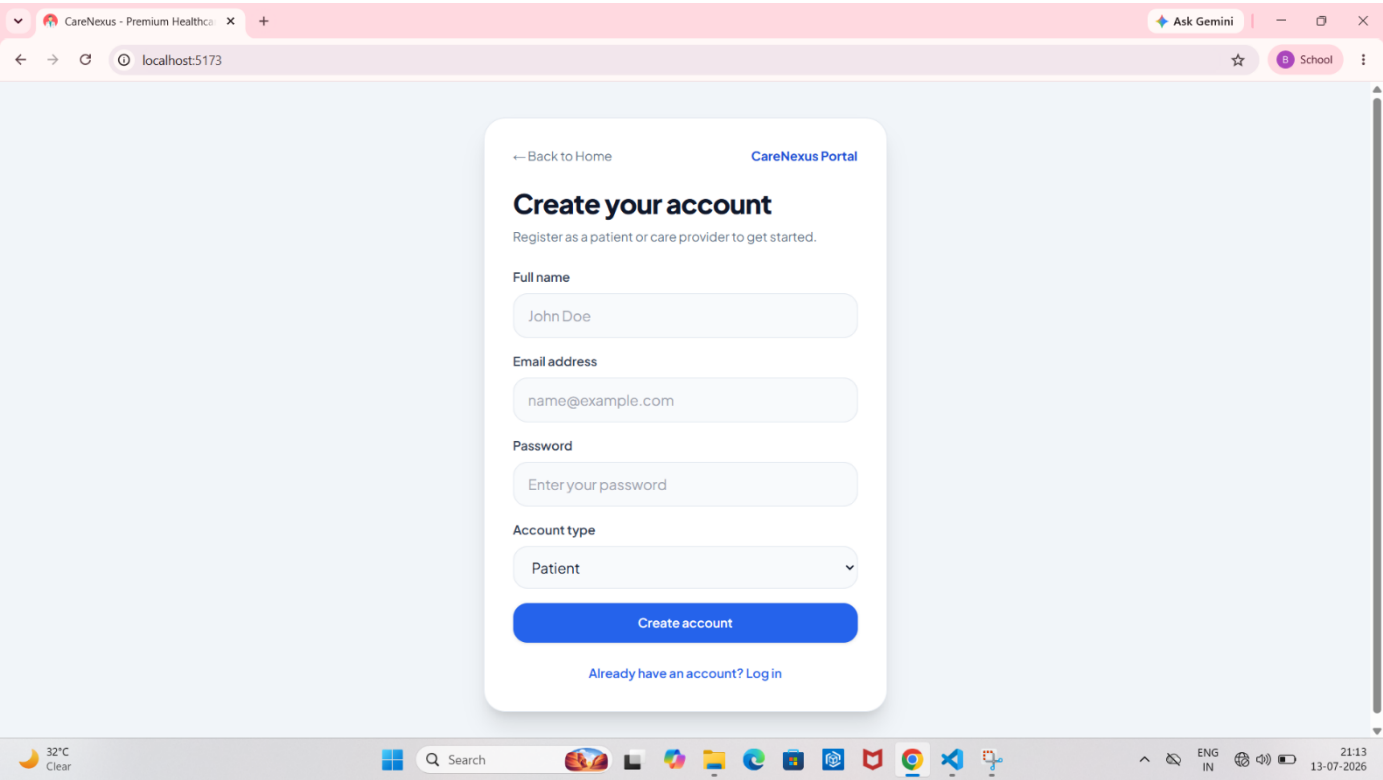
Login Portal

The login portal provides a card interface for user authentication. It validates email inputs and password credentials, routing users to their respective dashboards on success.

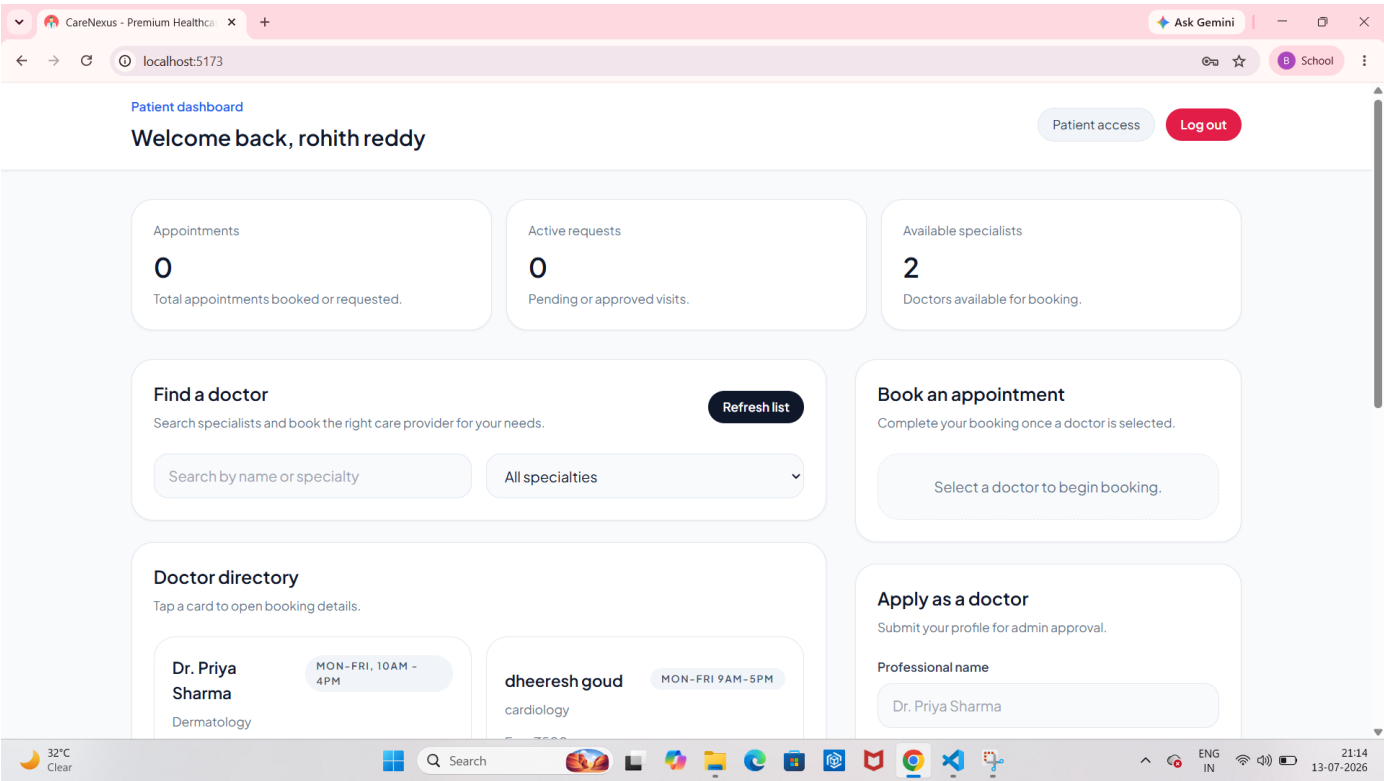


Register Portal

The registration screen allows new patients and doctors to create accounts, featuring a dropdown selector to choose their initial role.



Patient Dashboard



The patient dashboard displays available doctors, specializing filters, calendar selectors for booking, and booking status history.

Doctor Dashboard: Appointment Requests Queue

The doctor dashboard displays appointment counters and the booking request queue, where practitioners can approve or decline incoming patient visits.

Doctor Dashboard

Welcome, Dr. Dr.Charan

Logout

Pending Requests

1

Approved

0

Completed

0

Consultation Fee

₹1000

Appointments

Profile

Appointment Requests

Dheeresh

Requested: Wed, Jun 17, 2026 at 09:00 AM

Pending

Approve

Decline

Upcoming Appointments

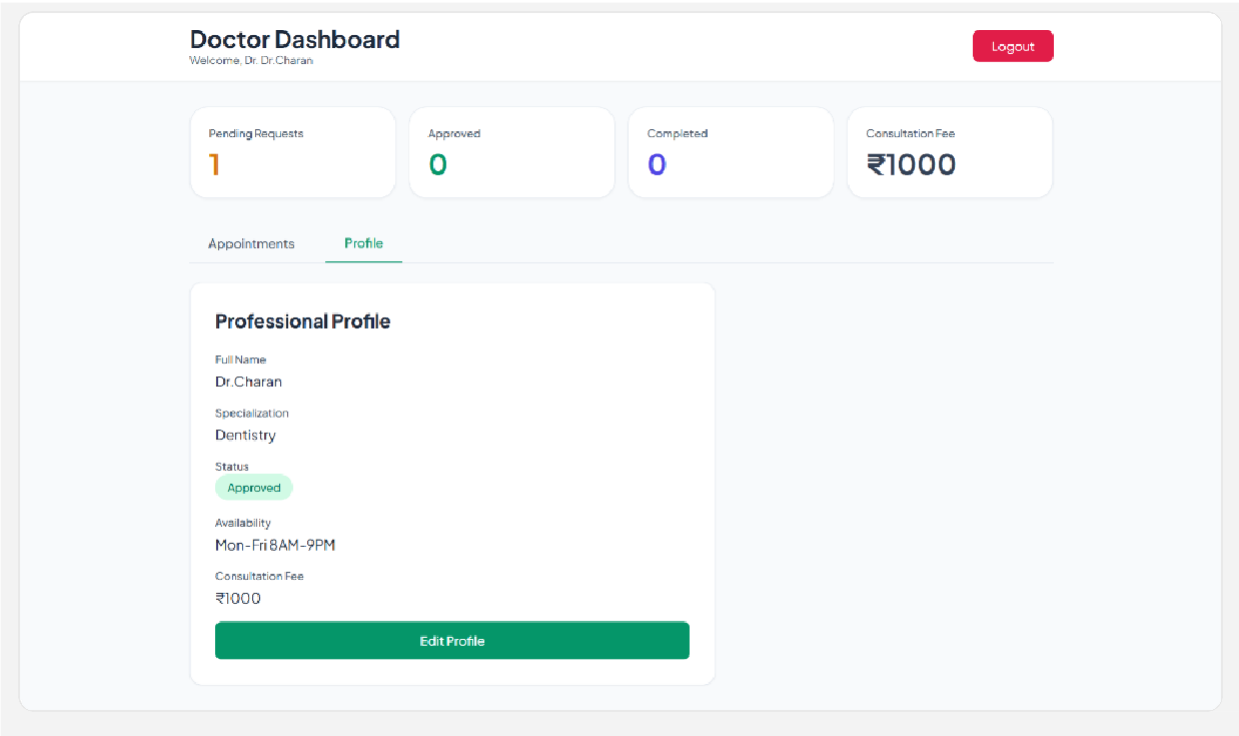
No upcoming appointments.

Appointment History

Patient	Date	Status
Dheeresh	Jun 17, 2026 at 09:00 AM	Pending

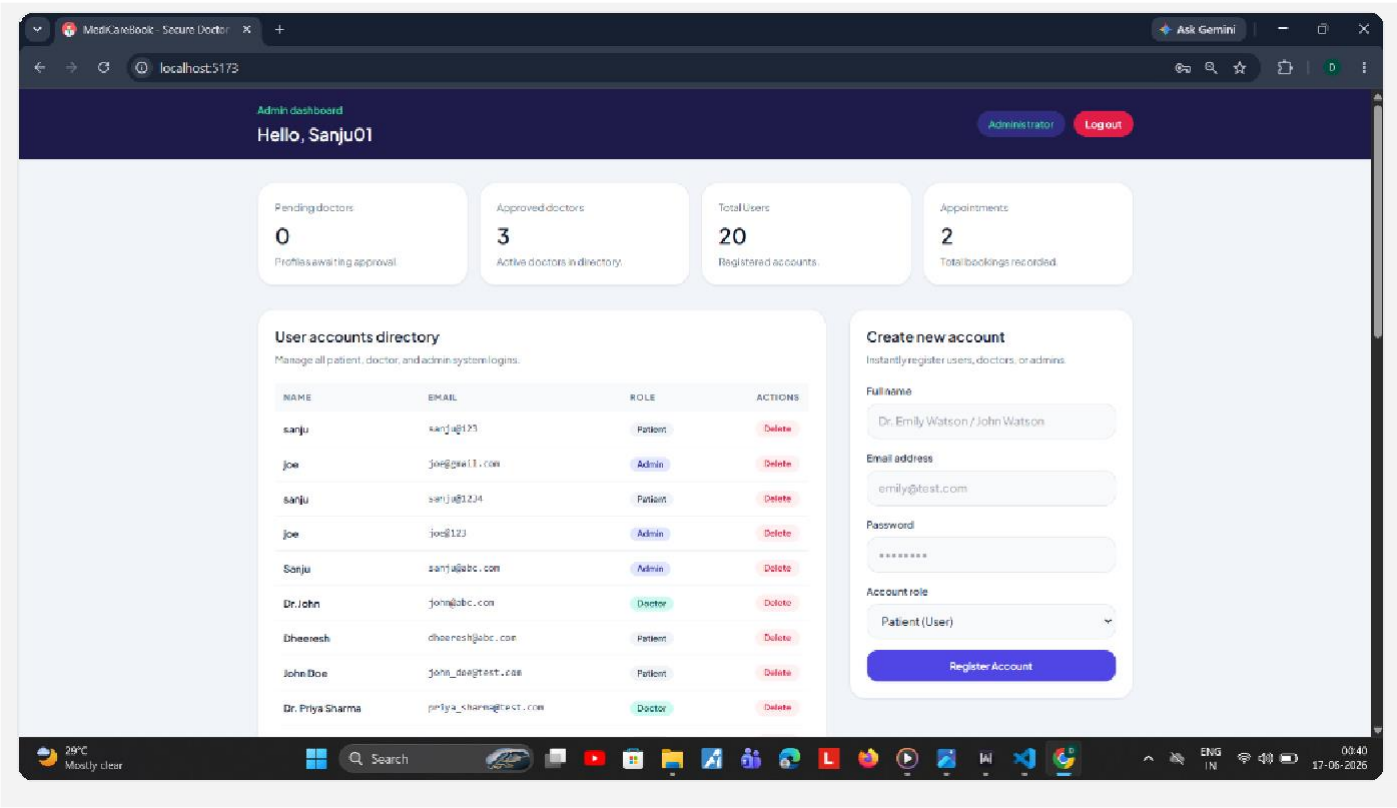
Doctor Dashboard: Professional Profile Manager

The profile tab allows practitioners to set their specialization details, edit consultation fees, and update active working hours.



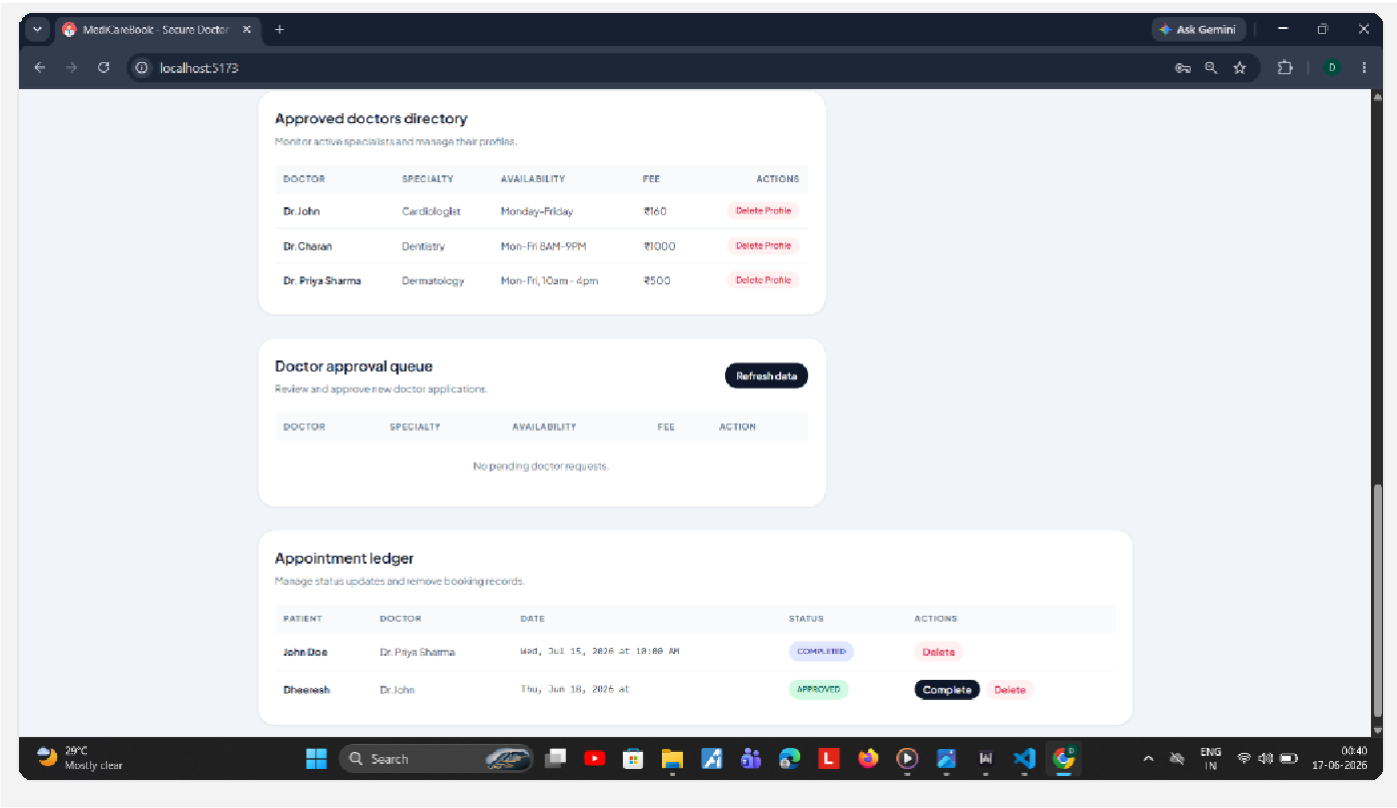
Administrative Dashboard: Overview & Registrations

The top panel of the admin dashboard displays platform-wide statistics, active user accounts, and registration forms to quickly add new users.



Administrative Dashboard: Directory Management & Ledgers

The lower panel of the admin dashboard displays the approved doctors list and the global appointment ledger. Admins can delete profiles, demote users back to patients, and update appointment states.



12. Known Issues

12.1 Local Storage State Synchronization

- **Problem:** When an admin approves a patient's doctor application, the patient's role changes in the database, but their dashboard view does not update immediately because their role is cached in the browser's local storage.
- **Workaround:** The user must log out and log back in to refresh their dashboard state.
- **Roadmap Resolution:** Implement an interceptor that polls the `/api/auth/getuserdata` route on page load, dynamically updating the browser session.

12.2 Global Axios Error Interceptors

- **Problem:** Connection drops between React and Express are handled locally inside components, which can result in inconsistent error UI.
- **Roadmap Resolution:** Configure a global Axios response interceptor inside `app.jsx` to display a system-wide error notification banner during connection issues.

13. Future Enhancements

13.1 Video Consultation & WebRTC Integration

Integrate PeerJS or WebRTC libraries to allow patients and doctors to start video calls directly from their dashboard panels. This will remove the need for external video conferencing apps, providing a unified medical consulting experience.

13.2 SMS Gateway & Push Notifications

Connect the appointment update routes to Twilio or a similar SMS gateway. This will trigger text reminders to patients when their appointment status updates (e.g., from `pending` to `approved` or `completed`).

13.3 Prescription PDF Generation & Export

Allow doctors to fill out a prescription details form on their dashboard. This form will generate a secure PDF document containing the doctor's signature and prescription details, which is automatically uploaded to the appointment's record for patient download.

Appendix A: Full Source Code Listings

A.1 Backend Setup & Server Execution

database connection logic (backend/config/db.js)

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB Connected successfully: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Database connection error: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Server entry point (backend/index.js)

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const connectDB = require('./config/db');

const authRoutes = require('./routes/authRoutes');
const doctorRoutes = require('./routes/doctorRoutes');

const app = express();

connectDB();

app.use(cors());
app.use(express.json());

app.use('/api/auth', authRoutes);
app.use('/api/doctors', doctorRoutes);

app.get('/', (req, res) => {
  res.send('Book A Doctor API is running successfully.');
```

```
});

app.use((err, req, res, next) => {
  console.error('Unhandled Server Error:', err.stack);
  res.status(500).json({ message: 'Something went wrong on the server!' });
});
```

```
const PORT = process.env.PORT || 8000;  
app.listen(PORT, () => {  
  console.log(`Server is running in dev mode on http://localhost:${PORT}`);  
});
```

A.2 Mongoose Schema Definitions

User Schema (`backend/models/User.js`)

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
    trim: true
  },
  email: {
    type: String,
    required: true,
    unique: true,
    trim: true,
    lowercase: true
  },
  password: {
    type: String,
    required: true
  },
  role: {
    type: String,
    enum: ['user', 'doctor', 'admin'],
    default: 'user'
  }
}, {
  timestamps: true
});

const User = mongoose.model('User', userSchema);
module.exports = User;
```

Doctor Schema (`backend/models/Doctor.js`)

```
const mongoose = require('mongoose');

const doctorSchema = new mongoose.Schema({
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  name: {
    type: String,
    required: true,
    trim: true
  }
});
```

```

    },
    specialization: {
      type: String,
      required: true,
      trim: true
    },
    availability: {
      type: String,
      required: true,
      trim: true
    },
    },
    fees: {
      type: Number,
      required: true
    },
    status: {
      type: String,
      enum: ['pending', 'approved'],
      default: 'pending'
    }
  }, {
    timestamps: true
  });

const Doctor = mongoose.model('Doctor', doctorSchema);
module.exports = Doctor;

```

Appointment Schema ([backend/models/Appointment.js](#))

```
const mongoose = require('mongoose');

const appointmentSchema = new mongoose.Schema({
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  doctorId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Doctor',
    required: true
  },
  userName: {
    type: String,
    required: true,
    trim: true
  },
  doctorName: {
    type: String,
    required: true,
    trim: true
  },
  date: {
    type: Date,
    required: true
  },
  time: {
    type: String,
    required: true,
    trim: true
  },
  status: {
    type: String,
    enum: ['pending', 'approved', 'completed', 'cancelled'],
    default: 'pending'
  }
}, {
  timestamps: true
});

const Appointment = mongoose.model('Appointment', appointmentSchema);
module.exports = Appointment;
```

A.3 Express Routing Interfaces

Auth Router (`backend/routes/authRoutes.js`)

```
const express = require('express');
const router = express.Router();
const authController = require('../controllers/authController');

router.post('/register', authController.register);
router.post('/login', authController.login);

router.get('/users', authController.listUsers);
router.put('/user/:userId', authController.updateUser);
router.delete('/user/:userId', authController.deleteUser);

module.exports = router;
```

Doctor Router (`backend/routes/doctorRoutes.js`)

```
const express = require('express');
const router = express.Router();
const doctorController = require('../controllers/doctorController');

router.post('/apply', doctorController.applyDoctor);
router.get('/list', doctorController.listDoctors);
router.get('/profile/user/:userId', doctorController.getDoctorProfileByUser);
router.put('/profile/:doctorId', doctorController.updateDoctorProfile);

router.post('/book', doctorController.bookAppointment);
router.get('/history/user/:userId', doctorController.getAppointmentsByUser);
router.get('/history/doctor/:doctorId', doctorController.getAppointmentsByDoctor);
router.get('/history/admin/all', doctorController.getAppointmentsAdmin);

router.put('/appointment/:appointmentId',
doctorController.updateAppointmentStatus);
router.put('/approve/:doctorId', doctorController.approveDoctor);
router.delete('/:doctorId', doctorController.deleteDoctor);
router.delete('/appointment/:appointmentId', doctorController.deleteAppointment);

module.exports = router;
```


A.4 MVC Business Logic Controllers

Auth Controller (`backend/controllers/authController.js`)

```
const User = require('../models/User');
const bcrypt = require('bcryptjs');

exports.register = async (req, res) => {
  try {
    const { name, email, password, role } = req.body;
    if (!name || !email || !password) {
      return res.status(400).json({ message: 'Please provide name, email, and password.' });
    }
    const userExists = await User.findOne({ email });
    if (userExists) {
      return res.status(400).json({ message: 'User with this email already exists.' });
    }
    const salt = await bcrypt.genSalt(10);
    const hashedPassword = await bcrypt.hash(password, salt);
    const newUser = new User({
      name,
      email,
      password: hashedPassword,
      role: role || 'user'
    });
    await newUser.save();
    return res.status(201).json({
      _id: newUser._id,
      name: newUser.name,
      email: newUser.email,
      role: newUser.role
    });
  } catch (error) {
    console.error('Registration error:', error);
    return res.status(500).json({ message: 'Internal server error during registration.' });
  }
};

exports.login = async (req, res) => {
  try {
    const { email, password } = req.body;
    if (!email || !password) {
      return res.status(400).json({ message: 'Please provide email and password.' });
    }
    const user = await User.findOne({ email });
    if (!user) {
      return res.status(400).json({ message: 'Invalid email or password.' });
    }
  }
};
```

```

    }
    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) {
        return res.status(400).json({ message: 'Invalid email or password.' });
    }
    return res.status(200).json({
        _id: user._id,
        name: user.name,
        email: user.email,
        role: user.role
    });
} catch (error) {
    console.error('Login error:', error);
    return res.status(500).json({ message: 'Internal server error during login.'
});
}
};

exports.listUsers = async (req, res) => {
    try {
        const users = await User.find({}).select('-password');
        return res.status(200).json(users);
    } catch (error) {
        return res.status(500).json({ message: 'Unable to retrieve users.' });
    }
};

exports.updateUser = async (req, res) => {
    try {
        const { userId } = req.params;
        const { name, email, role } = req.body;
        const user = await User.findById(userId);
        if (!user) {
            return res.status(404).json({ message: 'User not found.' });
        }
        if (name) user.name = name;
        if (email) user.email = email;
        if (role) user.role = role;
        await user.save();
        return res.status(200).json({
            _id: user._id,
            name: user.name,
            email: user.email,
            role: user.role
        });
    } catch (error) {
        return res.status(500).json({ message: 'Failed to update user.' });
    }
};

exports.deleteUser = async (req, res) => {
    try {
        const { userId } = req.params;
        const user = await User.findByIdAndDelete(userId);

```

```
if (!user) {
  return res.status(404).json({ message: 'User not found.' });
}
const Doctor = require('../models/Doctor');
const Appointment = require('../models/Appointment');
await Doctor.deleteOne({ userId });
await Appointment.deleteMany({
  $or: [{ userId }, { doctorId: userId }]
});
return res.status(200).json({ message: 'User and associated data deleted successfully.' });
} catch (error) {
  return res.status(500).json({ message: 'Failed to delete user.' });
}
};
```

Doctor Controller (backend/controllers/doctorController.js)

```
const Doctor = require('../models/Doctor');
const User = require('../models/User');
const Appointment = require('../models/Appointment');
const bcrypt = require('bcryptjs');

// Apply to become a doctor
exports.applyDoctor = async (req, res) => {
  try {
    const { userId, name, specialization, availability, fees } = req.body;

    let targetUserId = userId;

    // If userId is not provided (e.g. from user dashboard), find or create a user
    // account for them
    if (!targetUserId) {
      // Find user by name
      let user = await User.findOne({ name });
      if (user) {
        targetUserId = user._id;
        // Update role to doctor since they are applying to be one
        user.role = 'doctor';
        await user.save();
      } else {
        // Create a placeholder user account so the doctor can log in later
        const email = name.toLowerCase().replace(/\s+/g, '') + '@doctor.com';
        const salt = await bcrypt.genSalt(10);
        const passwordHash = await bcrypt.hash('123456', salt); // Default
        password for applications

        user = new User({
          name,
          email,
          password: passwordHash,
          role: 'doctor'
        });
        await user.save();
        targetUserId = user._id;
      }
    } else {
      // If userId is provided, make sure their role is updated to doctor
      const user = await User.findById(targetUserId);
      if (user && user.role !== 'doctor') {
        user.role = 'doctor';
        await user.save();
      }
    }

    // Check if doctor profile already exists for this user
    let doctor = await Doctor.findOne({ userId: targetUserId });
```

```

    if (doctor) {
      // Update existing application
      doctor.name = name;
      doctor.specialization = specialization;
      doctor.availability = availability;
      doctor.fees = fees;
      doctor.status = 'pending'; // Reset status to pending for admin re-approval
      await doctor.save();
    } else {
      // Create new doctor profile
      doctor = new Doctor({
        userId: targetUserId,
        name,
        specialization,
        availability,
        fees,
        status: 'pending'
      });
      await doctor.save();
    }

    return res.status(201).json(doctor);

  } catch (error) {
    console.error('Apply doctor error:', error);
    return res.status(500).json({ message: 'Unable to submit doctor application.'
  });
}

// List all doctors (pending and approved)
exports.listDoctors = async (req, res) => {
  try {
    const doctors = await Doctor.find({});
    return res.status(200).json(doctors);
  } catch (error) {
    console.error('List doctors error:', error);
    return res.status(500).json({ message: 'Unable to fetch doctors list.' });
  }
};

// Get doctor profile by userId
exports.getDoctorProfileByUser = async (req, res) => {
  try {
    const { userId } = req.params;
    const doctor = await Doctor.findOne({ userId });

    if (!doctor) {
      // Frontend expects a 404 error if profile does not exist to show creation
form
      return res.status(404).json({ message: 'Doctor profile not found.' });
    }

    return res.status(200).json(doctor);
  }
};

```

```

    } catch (error) {
      console.error('Get profile error:', error);
      return res.status(500).json({ message: 'Unable to load profile.' });
    }
  };

// Update doctor profile (availability, fees)
exports.updateDoctorProfile = async (req, res) => {
  try {
    const { doctorId } = req.params;
    const { availability, fees } = req.body;

    const doctor = await Doctor.findById(doctorId);
    if (!doctor) {
      return res.status(404).json({ message: 'Doctor profile not found.' });
    }

    doctor.availability = availability;
    doctor.fees = fees;
    await doctor.save();

    return res.status(200).json(doctor);
  } catch (error) {
    console.error('Update profile error:', error);
    return res.status(500).json({ message: 'Unable to update profile.' });
  }
};

// Book an appointment
exports.bookAppointment = async (req, res) => {
  try {
    const { userId, doctorId, userName, doctorName, date, time } = req.body;

    if (!userId || !doctorId || !userName || !doctorName || !date || !time) {
      return res.status(400).json({ message: 'Missing required fields for booking.' });
    }

    const appointment = new Appointment({
      userId,
      doctorId,
      userName,
      doctorName,
      date: new Date(date),
      time,
      status: 'pending'
    });

    await appointment.save();
    return res.status(201).json(appointment);
  } catch (error) {
    console.error('Book appointment error:', error);
    return res.status(500).json({ message: 'Booking failed. Please try again.' });
  }
}

```

```

};

// Get appointment history for a patient
exports.getAppointmentsByUser = async (req, res) => {
  try {
    const { userId } = req.params;
    const history = await Appointment.find({ userId }).sort({ createdAt: -1 });
    return res.status(200).json(history);
  } catch (error) {
    console.error('Get user history error:', error);
    return res.status(500).json({ message: 'Unable to retrieve appointments.' });
  }
};

// Get appointment history for a doctor
exports.getAppointmentsByDoctor = async (req, res) => {
  try {
    const { doctorId } = req.params;
    const history = await Appointment.find({ doctorId }).sort({ createdAt: -1 });
    return res.status(200).json(history);
  } catch (error) {
    console.error('Get doctor history error:', error);
    return res.status(500).json({ message: 'Unable to retrieve appointments.' });
  }
};

// Get all appointments in the system (Admin feature)
exports.getAppointmentsAdmin = async (req, res) => {
  try {
    const history = await Appointment.find({}).sort({ createdAt: -1 });
    return res.status(200).json(history);
  } catch (error) {
    console.error('Get admin history error:', error);
    return res.status(500).json({ message: 'Unable to retrieve all appointments.' });
  }
};

// Update appointment status (approve, decline/cancel, complete)
exports.updateAppointmentStatus = async (req, res) => {
  try {
    const { appointmentId } = req.params;
    const { status } = req.body;

    const appointment = await Appointment.findById(appointmentId);
    if (!appointment) {
      return res.status(404).json({ message: 'Appointment not found.' });
    }

    appointment.status = status;
    await appointment.save();

    return res.status(200).json(appointment);
  } catch (error) {

```

```

        console.error('Update appointment status error:', error);
        return res.status(500).json({ message: 'Unable to update appointment.' });
    }
};

// Approve doctor profile (Admin feature)
exports.approveDoctor = async (req, res) => {
    try {
        const { doctorId } = req.params;

        const doctor = await Doctor.findById(doctorId);
        if (!doctor) {
            return res.status(404).json({ message: 'Doctor profile not found.' });
        }

        doctor.status = 'approved';
        await doctor.save();

        // Ensure the associated user has the doctor role
        const user = await User.findById(doctor.userId);
        if (user && user.role !== 'doctor') {
            user.role = 'doctor';
            await user.save();
        }

        return res.status(200).json({ message: 'Doctor approved successfully.', doctor });
    } catch (error) {
        console.error('Approve doctor error:', error);
        return res.status(500).json({ message: 'Failed to approve doctor.' });
    }
};

// Delete a doctor profile (Admin feature)
exports.deleteDoctor = async (req, res) => {
    try {
        const { doctorId } = req.params;
        const doctor = await Doctor.findByIdAndDelete(doctorId);
        if (!doctor) {
            return res.status(404).json({ message: 'Doctor profile not found.' });
        }

        // Demote the associated user back to patient/user role
        const user = await User.findById(doctor.userId);
        if (user && user.role === 'doctor') {
            user.role = 'user';
            await user.save();
        }

        // Delete appointments for this doctor profile
        await Appointment.deleteMany({ doctorId });

        return res.status(200).json({ message: 'Doctor profile deleted successfully.' });
    }
};

```



```
    } catch (error) {
      console.error('Delete doctor error:', error);
      return res.status(500).json({ message: 'Failed to delete doctor profile.' });
    }
  };

// Delete an appointment (Admin / User feature)
exports.deleteAppointment = async (req, res) => {
  try {
    const { appointmentId } = req.params;
    const appointment = await Appointment.findByIdAndDelete(appointmentId);
    if (!appointment) {
      return res.status(404).json({ message: 'Appointment not found.' });
    }

    return res.status(200).json({ message: 'Appointment deleted successfully.' });
  } catch (error) {
    console.error('Delete appointment error:', error);
    return res.status(500).json({ message: 'Failed to delete appointment.' });
  }
};
```

A.5 Frontend Routing Bootstrapper

React entry script (frontend/src/main.jsx)

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './app.jsx'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

Application session controller (frontend/src/app.jsx)

```
import React, { useState, useEffect } from 'react';
import LandingPage from './components/LandingPage';
import LoginRegister from './components/LoginRegister';
import UserDashboard from './components/UserDashboard';
import DoctorDashboard from './components/DoctorDashboard';
import AdminDashboard from './components/AdminDashboard';

export default function App() {
  const [user, setUser] = useState(null);
  const [view, setView] = useState('landing');

  useEffect(() => {
    const savedUser = localStorage.getItem('user');
    if (savedUser) {
      const parsed = JSON.parse(savedUser);
      setUser(parsed);
      setView('dashboard');
    }
  }, []);

  const handleLogin = (userData) => {
    setUser(userData);
    localStorage.setItem('user', JSON.stringify(userData));
    setView('dashboard');
  };

  const handleLogout = () => {
    setUser(null);
    localStorage.removeItem('user');
    setView('landing');
  };

  if (view === 'landing') {
```

```
    return <LoginPage onNavigate={() => setView('auth')} />;
  }

  if (view === 'auth') {
    return <LoginRegister onLogin={handleLogin} onBack={() => setView('landing')}
/>;
  }

  if (view === 'dashboard' && user) {
    if (user.role === 'admin') {
      return <AdminDashboard user={user} onLogout={handleLogout} />;
    }
    if (user.role === 'doctor') {
      return <DoctorDashboard user={user} onLogout={handleLogout} />;
    }
    return <UserDashboard user={user} onLogout={handleLogout} />;
  }

  return <LoginPage onNavigate={() => setView('auth')} />;
}
```

A.6 Frontend Components & Dashboards

Landing Page Component (frontend/src/components/LandingPage.jsx)

```
import React from 'react';

export default function LandingPage({ onNavigate }) {
  return (
    <div className="min-h-screen bg-slate-900 text-white flex flex-col justify-between">
      { /* Navbar */ }
      <nav className="border-b border-slate-800 bg-slate-900/50 backdrop-blur sticky top-0 z-50">
        <div className="max-w-7xl mx-auto px-6 py-4 flex items-center justify-between">
          <div className="flex items-center gap-2">
            <span className="h-6 w-6 rounded-full bg-emerald-500 animate-pulse">
</span>
            <span className="font-bold text-xl tracking-wider bg-gradient-to-r from-emerald-400 to-teal-400 bg-clip-text text-transparent">
              MediCareBook
            </span>
          </div>
          <button
            onClick={onNavigate}
            className="px-6 py-2 bg-emerald-600 text-white rounded-full font-semibold hover:bg-emerald-500 transition shadow-lg shadow-emerald-950/20"
          >
            Access Portal
          </button>
        </div>
      </nav>

      { /* Hero */ }
      <main className="flex-1 flex items-center justify-center px-6 py-20 bg-[radial-gradient(ellipse_at_top,var(--tw-gradient-stops))] from-slate-800 via-slate-950 to-slate-950">
        <div className="max-w-4xl text-center space-y-8">
          <div className="inline-flex items-center gap-2 px-4 py-1.5 rounded-full border border-emerald-500/20 bg-emerald-500/5 text-emerald-400 text-xs font-semibold">
            <span>🌟</span> MERN-Stack Healthcare Ecosystem
          </div>
          <h1 className="text-5xl md:text-6xl font-black tracking-tight leading-none bg-gradient-to-b from-white to-slate-400 bg-clip-text text-transparent">
            Secure Scheduling & <br/>
            Verified Practitioner Directories
          </h1>
          <p className="text-slate-400 text-lg md:text-xl max-w-2xl mx-auto font-medium">
            MediCareBook streamlines appointment booking with strict

```

administrative controls, role-based security, and real-time practitioner schedules.

```

    </p>
    <div className="flex flex-col sm:flex-row gap-4 justify-center pt-4">
      <button
        onClick={onNavigate}
        className="px-8 py-4 bg-emerald-600 hover:bg-emerald-500 text-white
font-bold rounded-2xl transition shadow-xl shadow-emerald-950/30 text-base"
      >
        Book an Appointment
      </button>
      <button
        onClick={onNavigate}
        className="px-8 py-4 bg-slate-800 hover:bg-slate-700 text-slate-200
font-bold rounded-2xl transition text-base"
      >
        Practitioner Registration
      </button>
    </div>
  </div>
</main>

  { /* Footer */}
  <footer className="border-t border-slate-800 py-6 text-center text-slate-500
text-sm bg-slate-950">
    <p>&copy; {new Date().getFullYear()} MediCareBook. Developed by Addla
Dheeresh Goud. All rights reserved.</p>
  </footer>
</div>
);
}
```

Auth Component (frontend/src/components/LoginRegister.jsx)

```
import React, { useState } from 'react';
import axios from 'ajax';

export default function LoginRegister({ onLogin, onBack }) {
  const [isLogin, setIsLogin] = useState(true);
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [role, setRole] = useState('user');
  const [message, setMessage] = useState('');
  const [loading, setLoading] = useState(false);

  const handleSubmit = async (e) => {
    e.preventDefault();
    setLoading(true);
    setMessage('');
    try {
      if (isLogin) {
        const res = await axios.post('http://localhost:8000/api/auth/login', {
          email, password });
        onLogin(res.data);
      } else {
        await axios.post('http://localhost:8000/api/auth/register', { name, email,
          password, role });
        setMessage('Registration successful! Switch to Login.');
```

```
        setIsLogin(true);
      }
    } catch (err) {
      setMessage(err.response?.data?.message || 'Transaction failed.');
```

```
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="min-h-screen flex items-center justify-center bg-slate-950 p-6
      relative overflow-hidden">
      { /* Background Orbs */ }
      <div className="absolute top-1/4 left-1/4 h-96 w-96 rounded-full bg-emerald-
        500/10 blur-[100px] pointer-events-none"></div>
      <div className="absolute bottom-1/4 right-1/4 h-96 w-96 rounded-full bg-
        teal-500/10 blur-[100px] pointer-events-none"></div>

      <div className="bg-slate-900 border border-slate-800 rounded-3xl p-8 max-w-
        md w-full shadow-2xl relative z-10 space-y-6">
        <div>
          <button
            onClick={onBack}
            className="text-slate-400 hover:text-white text-sm font-semibold flex
```

```

items-center gap-1 transition"
  >
    &larr; Back to Home
  </button>
  <h2 className="text-3xl font-black text-white mt-4">{isLoggedIn ? 'Welcome
Back' : 'Get Started'}</h2>
  <p className="text-slate-400 text-sm mt-1">
    {isLoggedIn ? 'Sign in to schedule doctor consultations' : 'Create an
account to book specialist appointments'}
  </p>
</div>

{message && (
  <p className={`p-4 rounded-xl text-sm border font-medium
${message.includes('successful') ? 'bg-emerald-950/30 border-emerald-800 text-
emerald-400' : 'bg-rose-950/30 border-rose-800 text-rose-400'}`}>
    {message}
  </p>
)}

<form onSubmit={handleSubmit} className="space-y-4">
  {!isLoggedIn && (
    <div>
      <label className="block text-xs font-bold uppercase tracking-wider
text-slate-400 mb-1.5">Full Name</label>
      <input
        type="text"
        placeholder="Dr. Emily Watson / John Watson"
        value={name}
        onChange={(e) => setName(e.target.value)}
        className="w-full px-4 py-3 rounded-xl bg-slate-950 border border-
slate-800 text-white outline-none focus:border-emerald-500 focus:ring-1
focus:ring-emerald-500 transition text-sm"
        required
      />
    </div>
  )}
  <div>
    <label className="block text-xs font-bold uppercase tracking-wider
text-slate-400 mb-1.5">Email Address</label>
    <input
      type="email"
      placeholder="watson@test.com"
      value={email}
      onChange={(e) => setEmail(e.target.value)}
      className="w-full px-4 py-3 rounded-xl bg-slate-950 border border-
slate-800 text-white outline-none focus:border-emerald-500 focus:ring-1
focus:ring-emerald-500 transition text-sm"
      required
    />
  </div>
  <div>
    <label className="block text-xs font-bold uppercase tracking-wider
text-slate-400 mb-1.5">Password</label>

```

```

        <input
          type="password"
          placeholder="....."
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          className="w-full px-4 py-3 rounded-xl bg-slate-950 border border-
slate-800 text-white outline-none focus:border-emerald-500 focus:ring-1
focus:ring-emerald-500 transition text-sm"
          required
        />
      </div>
      {!isLogin && (
        <div>
          <label className="block text-xs font-bold uppercase tracking-wider
text-slate-400 mb-1.5">Account Privilege</label>
          <select
            value={role}
            onChange={(e) => setRole(e.target.value)}
            className="w-full px-4 py-3 rounded-xl bg-slate-950 border border-
slate-800 text-white outline-none focus:border-emerald-500 focus:ring-1
focus:ring-emerald-500 transition text-sm"
          >
            <option value="user">Patient (User)</option>
            <option value="doctor">Doctor Application</option>
            <option value="admin">Platform Admin</option>
          </select>
        </div>
      )}
      <button
        type="submit"
        disabled={loading}
        className="w-full bg-emerald-600 hover:bg-emerald-500 text-white py-3
rounded-xl font-bold transition shadow-lg shadow-emerald-950/20 text-sm"
      >
        {loading ? 'Processing...' : isLogin ? 'Sign In' : 'Sign Up'}
      </button>
    </form>

    <button
      onClick={() => setIsLogin(!isLogin)}
      className="w-full text-center text-sm font-semibold text-slate-400
hover:text-white transition mt-4"
    >
      {isLogin ? "New to MedicareBook? Create account" : "Already registered?
Access portal"}
    </button>
  </div>
</div>
);
}

```


Patient Dashboard Component (frontend/src/components/UserDashboard.jsx)

```
import React, { useEffect, useState } from 'react';
import axios from 'ajax';

const DAYS = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun'];
const TIMES = [
  '8AM', '9AM', '10AM', '11AM', '12PM', '1PM', '2PM', '3PM', '4PM', '5PM', '6PM',
  '7PM', '8PM', '9PM'
];
const APPOINTMENT_TIMES = [
  '08:00 AM', '09:00 AM', '10:00 AM', '11:00 AM', '12:00 PM', '01:00 PM', '02:00 PM',
  '03:00 PM', '04:00 PM', '05:00 PM', '06:00 PM', '07:00 PM', '08:00 PM'
];

function UserDashboard({ user, onLogout }) {
  const [doctors, setDoctors] = useState([]);
  const [history, setHistory] = useState([]);
  const [selectedDoctor, setSelectedDoctor] = useState(null);
  const [appointmentDate, setAppointmentDate] = useState('');
  const [appointmentTime, setAppointmentTime] = useState('09:00 AM');
  const [searchText, setSearchText] = useState('');
  const [specializationFilter, setSpecializationFilter] = useState('all');
  const [doctorName, setDoctorName] = useState('');
  const [specialization, setSpecialization] = useState('');

  // Structured availability selectors for apply form
  const [applyStartDay, setApplyStartDay] = useState('Mon');
  const [applyEndDay, setApplyEndDay] = useState('Fri');
  const [applyStartTime, setApplyStartTime] = useState('9AM');
  const [applyEndTime, setApplyEndTime] = useState('5PM');

  const [fee, setFee] = useState('');
  const [message, setMessage] = useState('');
  const [loading, setLoading] = useState(false);

  const refreshData = async () => {
    try {
      const [doctorResponse, historyResponse] = await Promise.all([
        axios.get('http://localhost:8000/api/doctors/list'),
        axios.get(`http://localhost:8000/api/doctors/history/user/${user._id}`),
      ]);
      setDoctors(doctorResponse.data);
      setHistory(historyResponse.data);
    } catch (error) {
      console.error(error);
      setMessage('Unable to load dashboard data. Please try again.');
```

```

    refreshData();
  }, []);

const filteredDoctors = doctors
  .filter((doctor) => doctor.status === 'approved')
  .filter((doctor) => {
    const matchesSearch = [doctor.name, doctor.specialization]
      .join(' ')
      .toLowerCase()
      .includes(searchText.toLowerCase());
    const matchesFilter = specializationFilter === 'all' ||
    doctor.specialization === specializationFilter;
    return matchesSearch && matchesFilter;
  });

const handleBooking = async (event) => {
  event.preventDefault();
  if (!selectedDoctor || !appointmentDate || !appointmentTime) {
    setMessage('Please select a doctor, date, and time before booking.');
```

return;

```

  }

  setLoading(true);
  setMessage('');

  try {
    await axios.post('http://localhost:8000/api/doctors/book', {
      userId: user._id,
      doctorId: selectedDoctor._id,
      userName: user.name,
      doctorName: selectedDoctor.name,
      date: appointmentDate,
      time: appointmentTime,
    });
    setMessage(`Appointment request sent to ${selectedDoctor.name}.`);
    setAppointmentDate('');
    setAppointmentTime('09:00 AM');
    setSelectedDoctor(null);
    refreshData();
  } catch (error) {
    console.error(error);
    setMessage('Booking failed. Please check your connection and try again.');
```

finally {

```

    setLoading(false);
  }
};

const handleCancel = async (appointmentId) => {
  setMessage('');

  try {
    await
    axios.put(`http://localhost:8000/api/doctors/appointment/${appointmentId}`, {
      status: 'cancelled',

```

```

    });
    setMessage('Appointment cancelled successfully.');
```

```

    refreshData();
  } catch (error) {
    console.error(error);
    setMessage('Unable to cancel appointment at this time.');
```

```

  }
};

const handleDoctorApplication = async (event) => {
  event.preventDefault();
  setLoading(true);
  setMessage('');

  try {
    // Format availability from day/time selectors (e.g. "Mon-Fri 9AM-5PM")
    const combinedAvailability = `${applyStartDay}-${applyEndDay}
    ${applyStartTime}-${applyEndTime}`;

    await axios.post('http://localhost:8000/api/doctors/apply', {
      userId: user._id,
      name: doctorName || user.name,
      specialization,
      availability: combinedAvailability,
      fees: fee,
    });
    setMessage('Doctor application submitted. Admin will review it soon.');
```

```

    setDoctorName('');
    setSpecialization('');
    setApplyStartDay('Mon');
    setApplyEndDay('Fri');
    setApplyStartTime('9AM');
    setApplyEndTime('5PM');
    setFee('');
    refreshData();
  } catch (error) {
    console.error(error);
    setMessage('Unable to submit doctor application.');
```

```

  } finally {
    setLoading(false);
  }
};

const upcomingAppointments = history.filter((entry) => entry.status ===
'pending' || entry.status === 'approved');
```

```

const specializations = [...new Set(doctors.filter((doctor) => doctor.status ===
'approved').map((doctor) => doctor.specialization))];

return (
  <div className="min-h-screen bg-slate-50 text-slate-900">
    <header className="bg-white border-b border-slate-200 shadow-sm">
      <div className="max-w-7xl mx-auto flex flex-col gap-4 px-6 py-5 lg:flex-
row lg:items-center lg:justify-between">
        <div>
```

```

        <p className="text-sm font-semibold text-emerald-600">Patient
dashboard</p>
        <h1 className="mt-1 text-2xl font-bold text-slate-800">Welcome,
{user.name}</h1>
    </div>
    <button
        type="button"
        onClick={onLogout}
        className="rounded-xl bg-rose-600 px-5 py-2 text-sm font-bold text-
white hover:bg-rose-700 transition"
    >
        Logout
    </button>
</div>
</header>

<main className="max-w-7xl mx-auto px-6 py-8 space-y-8">
    {message && (
        <div className={`rounded-2xl border px-5 py-4 text-sm font-medium
${message.includes('submitted') || message.includes('success')} ? 'border-emerald-
200 bg-emerald-50 text-emerald-800' : 'border-rose-200 bg-rose-50 text-rose-
800'}`}>
            {message}
        </div>
    )}

    <div className="grid gap-8 xl:grid-cols-[2fr_1fr]">
        <div className="space-y-8">
            {/* Search & Filter */}
            <div className="rounded-3xl border border-slate-200 bg-white p-6
shadow-sm space-y-4">
                <h2 className="text-xl font-bold text-slate-800">Find a
Specialist</h2>
                <div className="flex flex-col sm:flex-row gap-4">
                    <input
                        type="text"
                        placeholder="Search by specialty or doctor name..."
                        value={searchText}
                        onChange={(e) => setSearchText(e.target.value)}
                        className="flex-1 rounded-xl border border-slate-200 px-4 py-3
outline-none focus:border-emerald-600 text-sm bg-slate-50"
                    />
                    <select
                        value={specializationFilter}
                        onChange={(e) => setSpecializationFilter(e.target.value)}
                        className="rounded-xl border border-slate-200 px-4 py-3 outline-
none focus:border-emerald-600 text-sm bg-slate-50 font-semibold text-slate-700"
                    >
                        <option value="all">All Specialties</option>
                        {specializations.map((spec) => (
                            <option key={spec} value={spec}>{spec}</option>
                        ))}
                    </select>
                </div>
            </div>

```

```

    </div>

    { /* Doctor Directory */}
    <div className="space-y-4">
      <h2 className="text-xl font-bold text-slate-800">Available
Specialists</h2>
      {filteredDoctors.length === 0 ? (
        <div className="rounded-3xl border border-slate-200 bg-white p-8
text-center text-slate-500">
          No specialists found matching your search.
        </div>
      ) : (
        <div className="grid gap-4 sm:grid-cols-2">
          {filteredDoctors.map((doctor) => (
            <div
              key={doctor._id}
              onClick={() => setSelectedDoctor(doctor)}
              className={`rounded-3xl border p-5 bg-white cursor-pointer
hover:border-emerald-600 hover:shadow-md transition relative flex flex-col
justify-between min-h-[160px] ${selectedDoctor?._id === doctor._id ? 'border-
emerald-600 ring-2 ring-emerald-500/20' : 'border-slate-200'}`}
            >
              <div>
                <div className="flex justify-between items-start">
                  <p className="font-bold text-lg text-slate-800">
{doctor.name}</p>
                  <span className="rounded-full bg-emerald-50 px-3 py-1
text-xs font-bold text-emerald-700">Verified</span>
                </div>
                <p className="text-sm font-semibold text-slate-500
capitalize mt-1">{doctor.specialization}</p>
                </div>
                <div className="mt-4 pt-4 border-t border-slate-100 flex
items-center justify-between text-xs font-semibold text-slate-600">
                  <span>🕒 Availability: {doctor.availability}</span>
                  <span className="text-emerald-600 text-sm font-bold">₹
{doctor.fees}</span>
                </div>
              </div>
            </div>
          )
        </div>
      )}
    </div>

    { /* Appointment Ledger */}
    <div className="rounded-3xl border border-slate-200 bg-white p-6
shadow-sm space-y-6">
      <h2 className="text-xl font-bold text-slate-800">Personal Booking
History</h2>
      {history.length === 0 ? (
        <p className="text-slate-500 text-center py-6">You have no
appointments yet.</p>
      ) : (
        <div className="overflow-x-auto">

```

```

<table className="w-full text-left border-collapse text-sm">
  <thead>
    <tr className="border-b bg-slate-50 text-slate-500 uppercase
tracking-wider text-xs font-bold">
      <th className="p-3">Doctor</th>
      <th className="p-3">Date</th>
      <th className="p-3">Time</th>
      <th className="p-3">Status</th>
      <th className="p-3 text-right">Actions</th>
    </tr>
  </thead>
  <tbody className="divide-y divide-slate-100 text-slate-700">
    {history.map((appt) => (
      <tr key={appt._id} className="hover:bg-slate-50/50">
        <td className="p-3 font-semibold text-slate-900">
{appt.doctorName}</td>
        <td className="p-3">{new
Date(appt.date).toLocaleDateString('en-US', { year: 'numeric', month: 'short',
day: 'numeric' })}</td>
        <td className="p-3 font-mono">{appt.time}</td>
        <td className="p-3">
          <span className={`inline-flex rounded-full px-2.5 py-
0.5 text-xs font-semibold capitalize ${appt.status === 'completed' ? 'bg-indigo-50
text-indigo-700 border border-indigo-200' : appt.status === 'approved' ? 'bg-
emerald-50 text-emerald-700 border border-emerald-200' : appt.status ===
'cancelled' ? 'bg-rose-50 text-rose-700 border border-rose-200' : appt.status ===
'amber-50 text-amber-700 border border-amber-200'}`}>
            {appt.status}
          </span>
        </td>
        <td className="p-3 text-right">
          {(appt.status === 'pending' || appt.status ===
'approved') && (
            <button
              type="button"
              onClick={() => handleCancel(appt._id)}
              className="rounded-full bg-rose-50 px-3 py-1 text-
xs font-semibold text-rose-600 hover:bg-rose-100 transition"
            >
              Cancel
            </button>
          )}
        </td>
      </tr>
    )})}
  </tbody>
</table>
</div>
</div>

<aside className="space-y-8">
  {/* Booking Form */}

```

```

{selectedDoctor && (
  <div className="rounded-3xl border border-slate-200 bg-white p-6
shadow-sm space-y-4">
    <h3 className="text-xl font-bold text-slate-800">Book
{selectedDoctor.name}</h3>
    <form onSubmit={handleBooking} className="space-y-4">
      <div>
        <label className="block text-xs font-bold uppercase tracking-
wider text-slate-400 mb-1.5">Consultation Date</label>
        <input
          type="date"
          required
          value={appointmentDate}
          onChange={(e) => setAppointmentDate(e.target.value)}
          className="w-full rounded-xl border border-slate-200 px-4
py-3 outline-none focus:border-emerald-600 text-sm bg-slate-50"
        />
      </div>
      <div>
        <label className="block text-xs font-bold uppercase tracking-
wider text-slate-400 mb-1.5">Preferred Time</label>
        <select
          value={appointmentTime}
          onChange={(e) => setAppointmentTime(e.target.value)}
          className="w-full rounded-xl border border-slate-200 px-4
py-3 outline-none focus:border-emerald-600 text-sm bg-slate-50 text-slate-700
font-semibold"
        >
          {APPOINTMENT_TIMES.map((time) => (
            <option key={time} value={time}>{time}</option>
          ))}
        </select>
      </div>
      <button
        type="submit"
        disabled={loading}
        className="w-full bg-emerald-600 hover:bg-emerald-500 text-
white py-3 rounded-xl font-bold transition text-sm disabled:opacity-70"
      >
        {loading ? 'Submitting request...' : 'Confirm Appointment'}
      </button>
      <button
        type="button"
        onClick={() => setSelectedDoctor(null)}
        className="w-full bg-slate-100 hover:bg-slate-200 text-slate-
700 py-3 rounded-xl font-semibold transition text-sm"
      >
        Clear selection
      </button>
    </form>
  </div>
)}

{/* Doctor Application Form */}

```

```

    <div className="rounded-3xl border border-slate-200 bg-white p-6
shadow-sm space-y-4">
      <h3 className="text-xl font-bold text-slate-800">Become a
Practitioner</h3>
      <p className="text-xs text-slate-500">Apply to list your credentials
and start accepting digital appointments. Admin verification required.</p>

      <form onSubmit={handleDoctorApplication} className="space-y-4">
        <div>
          <label className="block text-xs font-bold uppercase tracking-
wider text-slate-400 mb-1.5">Professional Name</label>
          <input
            type="text"
            required
            value={doctorName}
            onChange={(e) => setDoctorName(e.target.value)}
            placeholder="Dr. Emily Watson"
            className="w-full rounded-xl border border-slate-200 px-4 py-3
outline-none focus:border-emerald-600 text-sm bg-slate-50"
          />
        </div>
        <div>
          <label className="block text-xs font-bold uppercase tracking-
wider text-slate-400 mb-1.5">Specialization</label>
          <input
            type="text"
            required
            value={specialization}
            onChange={(e) => setSpecialization(e.target.value)}
            placeholder="Pediatrics, Neurology, etc."
            className="w-full rounded-xl border border-slate-200 px-4 py-3
outline-none focus:border-emerald-600 text-sm bg-slate-50"
          />
        </div>
        <div>
          <label className="block text-xs font-bold uppercase tracking-
wider text-slate-400 mb-1.5">Availability Days</label>
          <div className="grid grid-cols-2 gap-2">
            <select
              value={applyStartDay}
              onChange={(e) => setApplyStartDay(e.target.value)}
              className="rounded-xl border border-slate-200 px-3 py-2
outline-none focus:border-emerald-600 text-xs bg-slate-50 text-slate-700 font-
semibold"
            >
              {DAYS.map(d => <option key={d} value={d}>{d}</option>)}
            </select>
            <select
              value={applyEndDay}
              onChange={(e) => setApplyEndDay(e.target.value)}
              className="rounded-xl border border-slate-200 px-3 py-2
outline-none focus:border-emerald-600 text-xs bg-slate-50 text-slate-700 font-
semibold"
            >

```



```

        {DAYS.map(d => <option key={d} value={d}>{d}</option>)}
      </select>
    </div>
  </div>
  <div>
    <label className="block text-xs font-bold uppercase tracking-
wider text-slate-400 mb-1.5">Availability Hours</label>
    <div className="grid grid-cols-2 gap-2">
      <select
        value={applyStartTime}
        onChange={(e) => setApplyStartTime(e.target.value)}
        className="rounded-xl border border-slate-200 px-3 py-2
outline-none focus:border-emerald-600 text-xs bg-slate-50 text-slate-700 font-
semibold"
      >
        {TIMES.map(t => <option key={t} value={t}>{t}</option>)}
      </select>
      <select
        value={applyEndTime}
        onChange={(e) => setApplyEndTime(e.target.value)}
        className="rounded-xl border border-slate-200 px-3 py-2
outline-none focus:border-emerald-600 text-xs bg-slate-50 text-slate-700 font-
semibold"
      >
        {TIMES.map(t => <option key={t} value={t}>{t}</option>)}
      </select>
    </div>
  </div>
  <div>
    <label className="block text-xs font-bold uppercase tracking-
wider text-slate-400 mb-1.5">Consultation Fee (₹)</label>
    <input
      type="number"
      required
      min="100"
      value={fee}
      onChange={(e) => setFee(e.target.value)}
      placeholder="1200"
      className="w-full rounded-xl border border-slate-200 px-4 py-3
outline-none focus:border-emerald-600 text-sm bg-slate-50"
    />
  </div>
  <button
    type="submit"
    disabled={loading}
    className="w-full bg-slate-900 hover:bg-slate-800 text-white py-3
rounded-xl font-bold transition text-sm disabled:opacity-75"
  >
    {loading ? 'Submitting application...' : 'Apply for Listing'}
  </button>
</form>
</div>
</aside>
</div>

```

```
        </main>
      </div>
    );
  }

  export default UserDashboard;
```

Doctor Dashboard Component (frontend/src/components/DoctorDashboard.jsx)

```
import React, { useState, useEffect } from 'react';
import axios from 'ajax';

const DAYS = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun'];
const TIMES = [
  '8AM', '9AM', '10AM', '11AM', '12PM', '1PM', '2PM', '3PM', '4PM', '5PM', '6PM',
  '7PM', '8PM', '9PM'
];

export default function DoctorDashboard({ user, onLogout }) {
  const [doctorProfile, setDoctorProfile] = useState(null);
  const [appointments, setAppointments] = useState([]);
  const [loading, setLoading] = useState(true);
  const [message, setMessage] = useState('');
  const [activeTab, setActiveTab] = useState('appointments');
  const [editMode, setEditMode] = useState(false);
  const [fees, setFees] = useState('');
  const [specializationForm, setSpecializationForm] = useState('');
  const [showApplicationForm, setShowApplicationForm] = useState(false);

  // Availability selector states
  const [startDay, setStartDay] = useState('Mon');
  const [endDay, setEndDay] = useState('Fri');
  const [startTime, setStartTime] = useState('9AM');
  const [endTime, setEndTime] = useState('5PM');

  useEffect(() => {
    refreshDoctorData();
  }, []);

  const refreshDoctorData = async () => {
    setLoading(true);
    try {
      const profileRes = await
    axios.get(`http://localhost:8000/api/doctors/profile/user/${user._id}`);
      const myProfile = profileRes.data;
      setDoctorProfile(myProfile);
      setFees(myProfile.fees);

      // Parse loaded availability (e.g. "Mon-Fri 9AM-5PM")
      if (myProfile.availability) {
        const parts = myProfile.availability.split(/[\s,]+/);
        if (parts.length >= 2) {
          const daysPart = parts[0];
          const hoursPart = parts[1];

          const days = daysPart.split('-');
          if (days.length === 2) {
            setStartDay(days[0]);

```

```

        setEndDay(days[1]);
    }

    const hours = hoursPart.split('-');
    if (hours.length === 2) {
        setStartTime(hours[0]);
        setEndTime(hours[1]);
    }
}
}

const appointmentsRes = await
axios.get(`http://localhost:8000/api/doctors/history/doctor/${myProfile._id}`);
setAppointments(appointmentsRes.data);
} catch (error) {
    if (error.response?.status === 404) {
        setDoctorProfile(null);
        setAppointments([]);
    } else {
        console.error(error);
        setMessage('Unable to load profile data. Please refresh.');
```

```

    } finally {
        setLoading(false);
    }
};
```

```

const handleUpdateProfile = async (e) => {
    e.preventDefault();
    if (!doctorProfile) {
        setMessage('Doctor profile not found.');
```

```

        return;
    }

    setLoading(true);
    setMessage('');
    try {
        const combinedAvailability = `${startDay}-${endDay}
${startTime}-${endTime}`;
```

```

        const updatedDoctor = await axios.put(
            `http://localhost:8000/api/doctors/profile/${doctorProfile._id}`,
            {
                availability: combinedAvailability,
                fees: parseFloat(fees),
            }
        );
```

```

        setDoctorProfile(updatedDoctor.data);
        setEditMode(false);
        setMessage('Profile updated successfully.');
```

```

        setTimeout(() => setMessage(''), 3000);
        refreshDoctorData();
    } catch (error) {
```

```

        console.error(error);
        setMessage('Unable to update profile. Please try again.');
```

} finally {

```

        setLoading(false);
    }
};

const handleApplyAsDoctor = async (e) => {
    e.preventDefault();
    if (!specializationForm || !fees) {
        setMessage('Please fill in all required fields.');
```

return;

```

    }

    setLoading(true);
    setMessage('');
    try {
        const combinedAvailability = `${startDay}-${endDay}
        ${startTime}-${endTime}`;

        const res = await axios.post('http://localhost:8000/api/doctors/apply', {
            userId: user._id,
            name: user.name,
            specialization: specializationForm.trim(),
            availability: combinedAvailability,
            fees: parseFloat(fees),
        });

        setDoctorProfile(res.data);
        setShowApplicationForm(false);
        setMessage('Application submitted successfully. Awaiting admin approval.');
```

setTimeout(() => setMessage(''), 3000);

```

        setSpecializationForm('');
        setStartDay('Mon');
        setEndDay('Fri');
        setStartTime('9AM');
        setEndTime('5PM');
        setFees('');
        refreshDoctorData();
    } catch (error) {
        console.error(error);
        setMessage('Unable to submit application. Please try again.');
```

} finally {

```

        setLoading(false);
    }
};

const handleUpdateAppointmentStatus = async (appointmentId, newStatus) => {
    setLoading(true);
    setMessage('');
    try {
        await axios.put(
            `http://localhost:8000/api/doctors/appointment/${appointmentId}`,
            { status: newStatus }
        );
    } catch (error) {
        console.error(error);
        setMessage('Unable to update appointment status. Please try again.');
```

} finally {

```

        setLoading(false);
    }
};

```



```

        </div>
        <button
            onClick={onLogout}
            className="px-6 py-2 bg-rose-600 text-white rounded-lg hover:bg-rose-
700 transition"
        >
            Logout
        </button>
    </div>
</header>

{/* Message */}
{message && (
    <div
        className={`max-w-6xl mx-auto mt-4 px-6 py-3 rounded-2xl border ${
            message.includes('successfully')
                ? 'bg-emerald-50 border-emerald-200 text-emerald-700'
                : 'bg-rose-50 border-rose-200 text-rose-700'
            }`}
    >
        {message}
    </div>
)}

<main className="max-w-6xl mx-auto px-6 py-8">
    {/* Stats Cards */}
    <div className="grid grid-cols-1 md:grid-cols-4 gap-4 mb-8">
        <div className="bg-white rounded-3xl p-6 shadow-sm border border-slate-
200">
            <p className="text-slate-600 text-sm font-medium">Pending Requests</p>
            <p className="text-4xl font-bold text-amber-600 mt-2">{stats.pending}</p>
        </div>
        <div className="bg-white rounded-3xl p-6 shadow-sm border border-slate-
200">
            <p className="text-slate-600 text-sm font-medium">Approved</p>
            <p className="text-4xl font-bold text-emerald-600 mt-2">
{stats.approved}</p>
        </div>
        <div className="bg-white rounded-3xl p-6 shadow-sm border border-slate-
200">
            <p className="text-slate-600 text-sm font-medium">Completed</p>
            <p className="text-4xl font-bold text-indigo-600 mt-2">
{stats.completed}</p>
        </div>
        <div className="bg-white rounded-3xl p-6 shadow-sm border border-slate-
200">
            <p className="text-slate-600 text-sm font-medium">Consultation Fee</p>
            <p className="text-4xl font-bold text-slate-700 mt-2">₹
{doctorProfile?.fees || 0}</p>
        </div>
    </div>

    {/* Tabs */}

```

```

<div className="flex gap-4 border-b border-slate-200 mb-6">
  {[ 'appointments', 'profile' ].map((tab) => (
    <button
      key={tab}
      onClick={() => setActiveTab(tab)}
      className={`px-6 py-3 font-medium transition ${
        activeTab === tab
          ? 'text-emerald-600 border-b-2 border-emerald-600'
          : 'text-slate-600 hover:text-slate-800'
      }`}
    >
      {tab === 'appointments' ? 'Appointments' : 'Profile'}
    </button>
  )]}
</div>

{/* Appointments Tab */}
{activeTab === 'appointments' && (
  <div className="space-y-6">
    {/* Pending Requests */}
    <div>
      <h2 className="text-xl font-bold text-slate-800 mb-4">Appointment
Requests</h2>
      {appointments.filter((a) => a.status === 'pending').length === 0 ? (
        <p className="text-slate-600">No pending requests.</p>
      ) : (
        <div className="space-y-3">
          {appointments
            .filter((a) => a.status === 'pending')
            .map((appt) => (
              <div
                key={appt._id}
                className="bg-white rounded-2xl p-6 border border-amber-
200 shadow-sm"
              >
                <div className="flex items-center justify-between mb-4">
                  <div>
                    <p className="font-semibold text-slate-800">
{appt.userName}</p>
                    <p className="text-sm text-slate-600">
                      Requested: {new
Date(appt.date).toLocaleDateString('en-US', {
                        weekday: 'short',
                        year: 'numeric',
                        month: 'short',
                        day: 'numeric',
                      })} at {appt.time}
                    </p>
                  </div>
                  <span className="px-3 py-1 bg-amber-100 text-amber-700
rounded-full text-sm font-medium">
                      Pending
                    </span>
                  </div>
                </div>
              </div>
            )
          }
        </div>
      )
    }
  )
}

```



```

        <div className="flex gap-3">
          <button
            onClick={() => handleUpdateAppointmentStatus(appt._id,
'approved')}}
            disabled={loading}
            className="flex-1 px-4 py-2 bg-emerald-600 text-white
rounded-lg hover:bg-emerald-700 transition disabled:opacity-50"
          >
            Approve
          </button>
          <button
            onClick={() => handleUpdateAppointmentStatus(appt._id,
'cancelled')}}
            disabled={loading}
            className="flex-1 px-4 py-2 bg-rose-600 text-white
rounded-lg hover:bg-rose-700 transition disabled:opacity-50"
          >
            Decline
          </button>
        </div>
      </div>
    )}
  </div>
)
</div>

{/* Upcoming Appointments */}
<div>
  <h2 className="text-xl font-bold text-slate-800 mb-4">Upcoming
Appointments</h2>
  {upcomingAppointments.filter((a) => a.status === 'approved').length
=== 0 ? (
    <p className="text-slate-600">No upcoming appointments.</p>
  ) : (
    <div className="space-y-3">
      {upcomingAppointments
        .filter((a) => a.status === 'approved')
        .map((appt) => (
          <div
            key={appt._id}
            className="bg-white rounded-2xl p-6 border border-emerald-
200 shadow-sm"
          >
            <div className="flex items-center justify-between mb-4">
              <div>
                <p className="font-semibold text-slate-800">
{appt.userName}</p>
                <p className="text-sm text-slate-600">
{new Date(appt.date).toLocaleDateString('en-US', {
  weekday: 'short',
  year: 'numeric',
  month: 'short',
  day: 'numeric',
})}} at {appt.time}

```

```

        </p>
      </div>
      <span className="px-3 py-1 bg-emerald-100 text-emerald-700 rounded-full text-sm font-medium">
        Approved
      </span>
    </div>
    <button
      onClick={() => handleUpdateAppointmentStatus(appt._id,
'completed')}}
      disabled={loading}
      className="w-full px-4 py-2 bg-indigo-600 text-white rounded-lg hover:bg-indigo-700 transition disabled:opacity-50"
    >
      Mark as Completed
    </button>
  </div>
  )))
</div>
})
</div>

{/* Appointment History Table */}
<div>
  <h2 className="text-xl font-bold text-slate-800 mb-4">Appointment
History</h2>
  {appointments.length === 0 ? (
    <p className="text-slate-600">No appointments yet.</p>
  ) : (
    <div className="overflow-x-auto rounded-2xl border border-slate-200">
      <table className="w-full text-sm">
        <thead className="bg-slate-100 border-b border-slate-200">
          <tr>
            <th className="px-6 py-3 text-left font-semibold text-slate-700">Patient</th>
            <th className="px-6 py-3 text-left font-semibold text-slate-700">Date</th>
            <th className="px-6 py-3 text-left font-semibold text-slate-700">Status</th>
          </tr>
        </thead>
        <tbody className="divide-y divide-slate-200">
          {appointments.map((appt) => (
            <tr key={appt._id} className="hover:bg-slate-50 transition">
              <td className="px-6 py-4">{appt.userName}</td>
              <td className="px-6 py-4">
                {new Date(appt.date).toLocaleDateString('en-US', {
                  month: 'short',
                  day: 'numeric',
                  year: 'numeric',
                })} at {appt.time}
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  )}
</div>

```

```
  |
```

```

        Availability Days
      </label>
      <div className="grid grid-cols-2 gap-4">
        <select
          value={startDay}
          onChange={(e) => setStartDay(e.target.value)}
          className="px-4 py-2 border border-slate-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-emerald-500"
        >
          {DAYS.map(d => <option key={d} value={d}>{d}</option>)}
        </select>
        <select
          value={endDay}
          onChange={(e) => setEndDay(e.target.value)}
          className="px-4 py-2 border border-slate-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-emerald-500"
        >
          {DAYS.map(d => <option key={d} value={d}>{d}</option>)}
        </select>
      </div>
    </div>
    <div>
      <label className="block text-sm font-medium text-slate-700
mb-2">
        Availability Hours
      </label>
      <div className="grid grid-cols-2 gap-4">
        <select
          value={startTime}
          onChange={(e) => setStartTime(e.target.value)}
          className="px-4 py-2 border border-slate-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-emerald-500"
        >
          {TIMES.map(t => <option key={t} value={t}>{t}</option>)}
        </select>
        <select
          value={endTime}
          onChange={(e) => setEndTime(e.target.value)}
          className="px-4 py-2 border border-slate-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-emerald-500"
        >
          {TIMES.map(t => <option key={t} value={t}>{t}</option>)}
        </select>
      </div>
    </div>
    <div>
      <label className="block text-sm font-medium text-slate-700
mb-2">
        Consultation Fee (₹)
      </label>
      <input
        type="number"
        required
        min="1"

```

```

        value={fees}
        onChange={(e) => setFees(e.target.value)}
        placeholder="Enter consultation fee"
        className="w-full px-4 py-2 border border-slate-300
rounded-lg focus:outline-none focus:ring-2 focus:ring-emerald-500"
      />
    </div>
    <div className="flex gap-3">
      <button
        type="submit"
        disabled={loading}
        className="flex-1 px-4 py-2 bg-emerald-600 text-white
rounded-lg hover:bg-emerald-700 transition disabled:opacity-50"
      >
        Submit Application
      </button>
      <button
        type="button"
        onClick={() => setShowApplicationForm(false)}
        className="flex-1 px-4 py-2 bg-slate-300 text-slate-800
rounded-lg hover:bg-slate-400 transition"
      >
        Cancel
      </button>
    </div>
  </form>
) : (
  <button
    onClick={() => setShowApplicationForm(true)}
    className="w-full px-4 py-3 bg-emerald-600 text-white rounded-
lg hover:bg-emerald-700 transition font-medium"
  >
    Start Application
  </button>
)}
</div>
) : (
  <div className="space-y-6">
    { /* Profile Info */ }
    <div className="bg-white rounded-2xl p-8 border border-slate-200
shadow-sm">
      <h2 className="text-2xl font-bold text-slate-800 mb-
6">Professional Profile</h2>

      <div className="space-y-4 mb-6">
        <div>
          <p className="text-sm font-medium text-slate-600">Full
Name</p>
          <p className="text-lg text-slate-800 mt-1">
{doctorProfile.name}</p>
        </div>
        <div>
          <p className="text-sm font-medium text-slate-
600">Specialization</p>

```

```

        <p className="text-lg text-slate-800 mt-1">
{doctorProfile.specialization}</p>
    </div>
    <div>
        <p className="text-sm font-medium text-slate-600">Status</p>
        <p className="mt-1">
            <span
                className={`px-4 py-2 rounded-full text-sm font-medium
                    ${
                        doctorProfile.status === 'approved'
                        ? 'bg-emerald-100 text-emerald-700'
                        : 'bg-amber-100 text-amber-700'
                    }`}
            >
                {doctorProfile.status === 'approved' ? 'Approved' :
'Pending Approval'}
            </span>
        </p>
    </div>
</div>

{editMode ? (
    <form onSubmit={handleUpdateProfile} className="space-y-4">
        <div>
            <label className="block text-sm font-medium text-slate-700
mb-2">
                Availability Days
            </label>
            <div className="grid grid-cols-2 gap-4">
                <select
                    value={startDay}
                    onChange={(e) => setStartDay(e.target.value)}
                    className="px-4 py-2 border border-slate-300 rounded-
lg focus:outline-none focus:ring-2 focus:ring-emerald-500"
                >
                    {DAYS.map(d => <option key={d} value={d}>{d}
</option>)}
                </select>
                <select
                    value={endDay}
                    onChange={(e) => setEndDay(e.target.value)}
                    className="px-4 py-2 border border-slate-300 rounded-
lg focus:outline-none focus:ring-2 focus:ring-emerald-500"
                >
                    {DAYS.map(d => <option key={d} value={d}>{d}
</option>)}
                </select>
            </div>
        </div>
        <div>
            <label className="block text-sm font-medium text-slate-700
mb-2">
                Availability Hours
            </label>

```

```

        <div className="grid grid-cols-2 gap-4">
          <select
            value={startTime}
            onChange={(e) => setStartTime(e.target.value)}
            className="px-4 py-2 border border-slate-300 rounded-
lg focus:outline-none focus:ring-2 focus:ring-emerald-500"
          >
            {TIMES.map(t => <option key={t} value={t}>{t}
</option>)}
          </select>
          <select
            value={endTime}
            onChange={(e) => setEndTime(e.target.value)}
            className="px-4 py-2 border border-slate-300 rounded-
lg focus:outline-none focus:ring-2 focus:ring-emerald-500"
          >
            {TIMES.map(t => <option key={t} value={t}>{t}
</option>)}
          </select>
        </div>
      </div>
      <div>
        <label className="block text-sm font-medium text-slate-700
mb-2">
          Consultation Fee (₹)
        </label>
        <input
          type="number"
          value={fees}
          onChange={(e) => setFees(e.target.value)}
          className="w-full px-4 py-2 border border-slate-300
rounded-lg focus:outline-none focus:ring-2 focus:ring-emerald-500"
        />
      </div>
      <div className="flex gap-3">
        <button
          type="submit"
          disabled={loading}
          className="flex-1 px-4 py-2 bg-emerald-600 text-white
rounded-lg hover:bg-emerald-700 transition disabled:opacity-50"
        >
          Save Changes
        </button>
        <button
          type="button"
          onClick={() => setEditMode(false)}
          className="flex-1 px-4 py-2 bg-slate-300 text-slate-800
rounded-lg hover:bg-slate-400 transition"
        >
          Cancel
        </button>
      </div>
    </form>
  ) : (

```

```

        <div className="space-y-4">
          <div>
            <p className="text-sm font-medium text-slate-600">Availability</p>
            <p className="text-lg text-slate-800 mt-1">{doctorProfile.availability}</p>
          </div>
          <div>
            <p className="text-sm font-medium text-slate-600">Consultation Fee</p>
            <p className="text-lg text-slate-800 mt-1">₹{doctorProfile.fees}</p>
          </div>
          <button
            onClick={() => setEditMode(true)}
            className="w-full mt-6 px-4 py-3 bg-emerald-600 text-white rounded-lg hover:bg-emerald-700 transition font-medium"
          >
            Edit Profile
          </button>
        </div>
      </div>
    </div>
  </div>
</main>
</div>
);
}

```


Admin Dashboard Component (frontend/src/components/AdminDashboard.jsx)

```
import React, { useEffect, useState } from 'react';
import axios from 'ajax';

function AdminDashboard({ user, onLogout }) {
  const [doctors, setDoctors] = useState([]);
  const [appointments, setAppointments] = useState([]);
  const [users, setUsers] = useState([]);
  const [message, setMessage] = useState('');

  // New user form states
  const [newUserName, setNewUserName] = useState('');
  const [newUserEmail, setNewUserEmail] = useState('');
  const [newUserPassword, setNewUserPassword] = useState('');
  const [newUserRole, setNewUserRole] = useState('user');
  const [createLoading, setCreateLoading] = useState(false);

  const refreshAdminData = async () => {
    try {
      const [doctorResponse, appointmentResponse, userResponse] = await
Promise.all([
        axios.get('http://localhost:8000/api/doctors/list'),
        axios.get('http://localhost:8000/api/doctors/history/admin/all'),
        axios.get('http://localhost:8000/api/auth/users'),
      ]);
      setDoctors(doctorResponse.data);
      setAppointments(appointmentResponse.data);
      setUsers(userResponse.data);
    } catch (error) {
      console.error(error);
      setMessage('Unable to load admin data.');
```

```

const updateAppointmentStatus = async (appointmentId, status) => {
  setMessage('');
  try {
    await
axios.put(`http://localhost:8000/api/doctors/appointment/${appointmentId}`, {
status });
    setMessage('Appointment status updated.');
```

```

    refreshAdminData();
  } catch (error) {
    console.error(error);
    setMessage('Unable to update appointment.');
```

```

  }
};

// Delete doctor profile
const handleDeleteDoctor = async (doctorId) => {
  if (!window.confirm('Are you sure you want to delete this doctor profile? This
will demote the user to patient and delete their appointments.')) return;
  setMessage('');
  try {
    await axios.delete(`http://localhost:8000/api/doctors/${doctorId}`);
    setMessage('Doctor profile deleted successfully.');
```

```

    refreshAdminData();
  } catch (error) {
    console.error(error);
    setMessage('Failed to delete doctor profile.');
```

```

  }
};

// Delete user account
const handleDeleteUser = async (userId) => {
  if (userId === user._id) {
    alert('You cannot delete your own admin account!');
    return;
  }
  if (!window.confirm('Are you sure you want to delete this user? This will
cascadingly delete all their appointments and profiles.')) return;
  setMessage('');
  try {
    await axios.delete(`http://localhost:8000/api/auth/user/${userId}`);
    setMessage('User deleted successfully.');
```

```

    refreshAdminData();
  } catch (error) {
    console.error(error);
    setMessage('Failed to delete user.');
```

```

  }
};

// Delete appointment record
const handleDeleteAppointment = async (appointmentId) => {
  if (!window.confirm('Are you sure you want to delete this appointment
record?')) return;
  setMessage('');
  try {

```

```

        await
    axios.delete(`http://localhost:8000/api/doctors/appointment/${appointmentId}`);
    setMessage('Appointment deleted successfully.');
```

refreshAdminData();

```

    } catch (error) {
        console.error(error);
        setMessage('Failed to delete appointment.');
```

}

```

    };

    // Create new user (patient / doctor / admin)
    const handleCreateUser = async (e) => {
        e.preventDefault();
        if (!newUserName || !newUserEmail || !newUserPassword) {
            alert('Please fill out all fields.');
```

return;

```

        }
        setCreateLoading(true);
        setMessage('');
        try {
            await axios.post('http://localhost:8000/api/auth/register', {
                name: newUserName,
                email: newUserEmail,
                password: newUserPassword,
                role: newUserRole,
            });
            setMessage(`Successfully created new ${newUserRole}: ${newUserName}`);
            setNewUserName('');
            setNewUserEmail('');
            setNewUserPassword('');
            setNewUserRole('user');
```

refreshAdminData();

```

        } catch (error) {
            console.error(error);
            setMessage(error.response?.data?.message || 'Failed to create user.');
```

finally {

```

            setCreateLoading(false);
        }
    };

    const pendingDoctors = doctors.filter((doctor) => doctor.status === 'pending');
    const approvedDoctorsList = doctors.filter((doctor) => doctor.status ===
    'approved');
```

return (

```

    <div className="min-h-screen bg-slate-100 text-slate-900">
      <header className="bg-indigo-950 text-white shadow-sm">
        <div className="max-w-7xl mx-auto flex flex-col gap-4 px-6 py-5 lg:flex-
row lg:items-center lg:justify-between">
          <div>
            <p className="text-sm font-semibold text-emerald-400">Admin
dashboard</p>
            <h1 className="mt-2 text-2xl font-semibold">Hello, {user.name}</h1>
          </div>
```

```

        <div className="flex flex-col gap-3 sm:flex-row sm:items-center">
            <span className="rounded-full bg-indigo-900 px-4 py-2 text-sm font-
semibold text-emerald-400">Administrator</span>
            <button
                type="button"
                onClick={onLogout}
                className="rounded-full bg-rose-600 px-4 py-2 text-sm font-semibold
text-white hover:bg-rose-700"
            >
                Log out
            </button>
        </div>
    </div>
</header>

<main className="max-w-7xl mx-auto px-6 py-8 space-y-8">
    {message && (
        <div className="rounded-3xl border border-emerald-200 bg-emerald-50 px-5
py-4 text-sm text-emerald-800 shadow-sm">
            {message}
        </div>
    )}

    {/* Admin stats */}
    <section className="grid gap-4 sm:grid-cols-4">
        <div className="rounded-3xl border border-slate-200 bg-white p-6 shadow-
sm">
            <p className="text-sm text-slate-500">Pending doctors</p>
            <p className="mt-3 text-3xl font-semibold text-slate-900">
{pendingDoctors.length}</p>
            <p className="mt-2 text-sm text-slate-500">Profiles awaiting approval.
</p>
        </div>
        <div className="rounded-3xl border border-slate-200 bg-white p-6 shadow-
sm">
            <p className="text-sm text-slate-500">Approved doctors</p>
            <p className="mt-3 text-3xl font-semibold text-slate-900">
{approvedDoctorsList.length}</p>
            <p className="mt-2 text-sm text-slate-500">Active doctors in
directory.</p>
        </div>
        <div className="rounded-3xl border border-slate-200 bg-white p-6 shadow-
sm">
            <p className="text-sm text-slate-500">Total Users</p>
            <p className="mt-3 text-3xl font-semibold text-slate-900">
{users.length}</p>
            <p className="mt-2 text-sm text-slate-500">Registered accounts.</p>
        </div>
        <div className="rounded-3xl border border-slate-200 bg-white p-6 shadow-
sm">
            <p className="text-sm text-slate-500">Appointments</p>
            <p className="mt-3 text-3xl font-semibold text-slate-900">
{appointments.length}</p>
            <p className="mt-2 text-sm text-slate-500">Total bookings recorded.

```

```

</p>
    </div>
</section>

<section className="grid gap-8 xl:grid-cols-[2fr_1fr]">
  <div className="space-y-8">
    {/* User List Panel */}
    <div className="rounded-3xl border border-slate-200 bg-white p-6 shadow-sm">
      <h2 className="text-xl font-semibold text-slate-900">User accounts directory</h2>
      <p className="mt-2 text-sm text-slate-500">Manage all patient, doctor, and admin system logins.</p>

      <div className="mt-6 overflow-x-auto">
        <table className="w-full text-left border-collapse text-sm">
          <thead className="bg-slate-50 text-slate-500 uppercase tracking-[0.08em] text-xs">
            <tr>
              <th className="p-3">Name</th>
              <th className="p-3">Email</th>
              <th className="p-3">Role</th>
              <th className="p-3 text-right">Actions</th>
            </tr>
          </thead>
          <tbody className="divide-y divide-slate-100 text-slate-700">
            {users.length === 0 ? (
              <tr>
                <td colspan="4" className="p-6 text-center text-slate-500">No users found.</td>
              </tr>
            ) : (
              users.map((item) => (
                <tr key={item._id}>
                  <td className="p-3 font-semibold text-slate-900">{item.name}</td>
                  <td className="p-3 font-mono">{item.email}</td>
                  <td className="p-3">
                    <span className={`inline-flex rounded-full px-2.5 py-0.5 text-xs font-semibold capitalize ${item.role === 'admin' ? 'bg-indigo-100 text-indigo-800' : item.role === 'doctor' ? 'bg-teal-100 text-teal-800' : 'bg-slate-100 text-slate-800'}`}>
                      {item.role === 'user' ? 'Patient' : item.role}
                    </span>
                  </td>
                  <td className="p-3 text-right">
                    {item._id !== user._id && (
                      <button
                        type="button"
                        onClick={() => handleDeleteUser(item._id)}
                        className="rounded-full bg-rose-50 px-3 py-1 text-xs font-semibold text-rose-600 hover:bg-rose-100 transition"
                      >
                        Delete

```

```

        </button>
      )}
    </td>
  </tr>
</tbody>
</table>
</div>
</div>

{ /* Approved Doctor Panel */}
<div className="rounded-3xl border border-slate-200 bg-white p-6
shadow-sm">
  <h2 className="text-xl font-semibold text-slate-900">Approved
doctors directory</h2>
  <p className="mt-2 text-sm text-slate-500">Monitor active
specialists and manage their profiles.</p>

  <div className="mt-6 overflow-x-auto">
    <table className="w-full text-left border-collapse text-sm">
      <thead className="bg-slate-50 text-slate-500 uppercase tracking-
[0.08em] text-xs">
        <tr>
          <th className="p-3">Doctor</th>
          <th className="p-3">Specialty</th>
          <th className="p-3">Availability</th>
          <th className="p-3">Fee</th>
          <th className="p-3 text-right">Actions</th>
        </tr>
      </thead>
      <tbody className="divide-y divide-slate-100 text-slate-700">
        {approvedDoctorsList.length === 0 ? (
          <tr>
            <td colspan="5" className="p-6 text-center text-slate-
500">No active doctors.</td>
          </tr>
        ) : (
          approvedDoctorsList.map((doctor) => (
            <tr key={doctor._id}>
              <td className="p-3 font-semibold text-slate-900">
{doctor.name}</td>

              <td className="p-3">{doctor.specialization}</td>
              <td className="p-3">{doctor.availability}</td>
              <td className="p-3">₹{doctor.fees}</td>
              <td className="p-3 text-right">
                <button
                  type="button"
                  onClick={() => handleDeleteDoctor(doctor._id)}
                  className="rounded-full bg-rose-50 px-3 py-1 text-xs
font-semibold text-rose-600 hover:bg-rose-100 transition"
                >
                  Delete Profile
                </button>
              </td>
            </tr>
          )
        )
      </tbody>
    </table>
  </div>

```

```

        </td>
      </tr>
    </tbody>
  </table>
</div>
</div>

{ /* Doctor approval queue */ }
<div className="rounded-3xl border border-slate-200 bg-white p-6 shadow-sm">
  <div className="flex flex-col gap-4 lg:flex-row lg:items-center lg:justify-between">
    <div>
      <h2 className="text-xl font-semibold text-slate-900">Doctor approval queue</h2>
      <p className="mt-2 text-sm text-slate-500">Review and approve new doctor applications.</p>
    </div>
    <button
      type="button"
      onClick={refreshAdminData}
      className="rounded-full bg-slate-900 px-4 py-2 text-sm font-semibold text-white hover:bg-slate-800"
    >
      Refresh data
    </button>
  </div>

  <div className="mt-6 overflow-x-auto">
    <table className="w-full text-left border-collapse text-sm">
      <thead className="bg-slate-50 text-slate-500 uppercase tracking-[0.08em] text-xs">
        <tr>
          <th className="p-3">Doctor</th>
          <th className="p-3">Specialty</th>
          <th className="p-3">Availability</th>
          <th className="p-3">Fee</th>
          <th className="p-3">Action</th>
        </tr>
      </thead>
      <tbody className="divide-y divide-slate-100 text-slate-700">
        {pendingDoctors.length === 0 ? (
          <tr>
            <td colspan="5" className="p-6 text-center text-slate-500">No pending doctor requests.</td>
          </tr>
        ) : (
          pendingDoctors.map((doctor) => (
            <tr key={doctor._id}>
              <td className="p-3 font-semibold text-slate-900">{doctor.name}</td>
              <td className="p-3">{doctor.specialization}</td>

```

```

        <td className="p-3">{doctor.availability}</td>
        <td className="p-3">₹{doctor.fees}</td>
        <td className="p-3">
          <button
            type="button"
            onClick={() => approveDoctor(doctor._id)}
            className="rounded-full bg-emerald-600 px-3 py-1
text-sm font-semibold text-white hover:bg-emerald-700"
          >
            Approve
          </button>
        </td>
      </tr>
    </tbody>
  </table>
</div>
</div>
</div>

<aside className="space-y-8">
  { /* Create New User Panel */ }
  <div className="rounded-3xl border border-slate-200 bg-white p-6
shadow-sm">
    <h2 className="text-xl font-semibold text-slate-900">Create new
account</h2>
    <p className="mt-2 text-sm text-slate-500">Instantly register users,
doctors, or admins.</p>

    <form onSubmit={handleCreateUser} className="mt-6 space-y-4">
      <div>
        <label className="block text-sm font-semibold text-slate-
700">Full name</label>
        <input
          type="text"
          required
          value={newUserName}
          onChange={(e) => setNewUserName(e.target.value)}
          placeholder="Dr. Emily Watson / John Watson"
          className="mt-2 w-full rounded-2xl border border-slate-200 bg-
slate-50 px-4 py-3 text-slate-900 outline-none focus:border-emerald-500"
        />
      </div>
      <div>
        <label className="block text-sm font-semibold text-slate-
700">Email address</label>
        <input
          type="email"
          required
          value={newUserEmail}
          onChange={(e) => setNewUserEmail(e.target.value)}
          placeholder="emily@test.com"
          className="mt-2 w-full rounded-2xl border border-slate-200 bg-

```



```

slate-50 px-4 py-3 text-slate-900 outline-none focus:border-emerald-500"
    />
  </div>
  <div>
    <label className="block text-sm font-semibold text-slate-
700">Password</label>
    <input
      type="password"
      required
      value={newUserPassword}
      onChange={(e) => setNewUserPassword(e.target.value)}
      placeholder="....."
      className="mt-2 w-full rounded-2xl border border-slate-200 bg-
slate-50 px-4 py-3 text-slate-900 outline-none focus:border-emerald-500"
    />
  </div>
  <div>
    <label className="block text-sm font-semibold text-slate-
700">Account role</label>
    <select
      value={newUserRole}
      onChange={(e) => setNewUserRole(e.target.value)}
      className="mt-2 w-full rounded-2xl border border-slate-200 bg-
slate-50 px-4 py-3 text-slate-900 outline-none focus:border-emerald-500"
    >
      <option value="user">Patient (User)</option>
      <option value="doctor">Doctor</option>
      <option value="admin">Admin</option>
    </select>
  </div>
  <button
    type="submit"
    disabled={createLoading}
    className="w-full rounded-2xl bg-indigo-600 px-4 py-3 text-sm
font-semibold text-white shadow hover:bg-indigo-700 disabled:opacity-75"
  >
    {createLoading ? 'Registering...' : 'Register Account'}
  </button>
</form>
</div>
</aside>
</section>

{/* Appointment ledger */}
<section className="rounded-3xl border border-slate-200 bg-white p-6
shadow-sm">
  <h2 className="text-xl font-semibold text-slate-900">Appointment
ledger</h2>
  <p className="mt-2 text-sm text-slate-500">Manage status updates and
remove booking records.</p>

  <div className="mt-6 overflow-x-auto">
    <table className="w-full text-left border-collapse text-sm">
      <thead className="bg-slate-50 text-slate-500 uppercase tracking-

```

```

[0.08em] text-xs">
    <tr>
      <th className="p-3">Patient</th>
      <th className="p-3">Doctor</th>
      <th className="p-3">Date</th>
      <th className="p-3">Status</th>
      <th className="p-3">Actions</th>
    </tr>
  </thead>
  <tbody className="divide-y divide-slate-100 text-slate-700">
    {appointments.length === 0 ? (
      <tr>
        <td colspan="5" className="p-6 text-center text-slate-500">No
appointments recorded.</td>
      </tr>
    ) : (
      appointments.map((appointment) => (
        <tr key={appointment._id}>
          <td className="p-3 font-semibold text-slate-900">
{appointment.userName}</td>
          <td className="p-3">{appointment.doctorName}</td>
          <td className="p-3 font-mono">
            {new Date(appointment.date).toLocaleDateString('en-US', {
              weekday: 'short',
              year: 'numeric',
              month: 'short',
              day: 'numeric'
            })} at {appointment.time}
          </td>
          <td className="p-3">
            <span className={`inline-flex rounded-full px-3 py-1 text-
[11px] font-semibold uppercase ${appointment.status === 'approved' ? 'bg-emerald-
100 text-emerald-800' : appointment.status === 'completed' ? 'bg-indigo-100 text-
indigo-800' : appointment.status === 'cancelled' ? 'bg-rose-100 text-rose-800' :
'bg-amber-100 text-amber-800'}`}>
              {appointment.status}
            </span>
          </td>
          <td className="p-3 flex flex-wrap gap-2">
            {appointment.status === 'pending' && (
              <button
                type="button"
                onClick={() =>
updateAppointmentStatus(appointment._id, 'approved')}
                className="rounded-full bg-indigo-600 px-3 py-1 text-
sm font-semibold text-white hover:bg-indigo-700"
              >
                Approve
              </button>
            )}
            {appointment.status === 'approved' && (
              <button
                type="button"
                onClick={() =>

```

```

updateAppointmentStatus(appointment._id, 'completed'))
      className="rounded-full bg-slate-900 px-3 py-1 text-sm
font-semibold text-white hover:bg-slate-800"
    >
      Complete
    </button>
  )}
  <button
    type="button"
    onClick={() => handleDeleteAppointment(appointment._id)}
    className="rounded-full bg-rose-50 px-3 py-1 text-sm
font-semibold text-rose-600 hover:bg-rose-100"
    >
      Delete
    </button>
  </td>
</tr>
))
)}
</tbody>
</table>
</div>
</section>
</main>
</div>
);
}

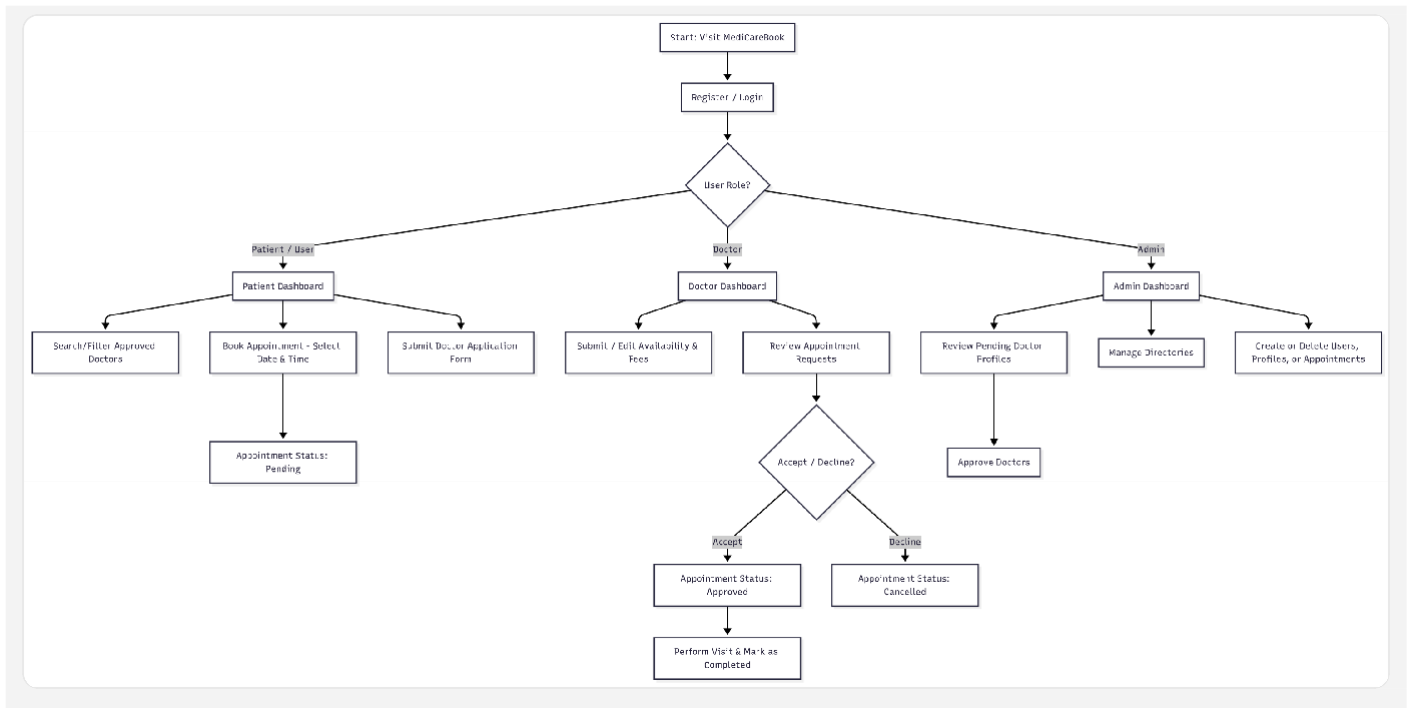
export default AdminDashboard;

```

Appendix B: Technical Diagrams

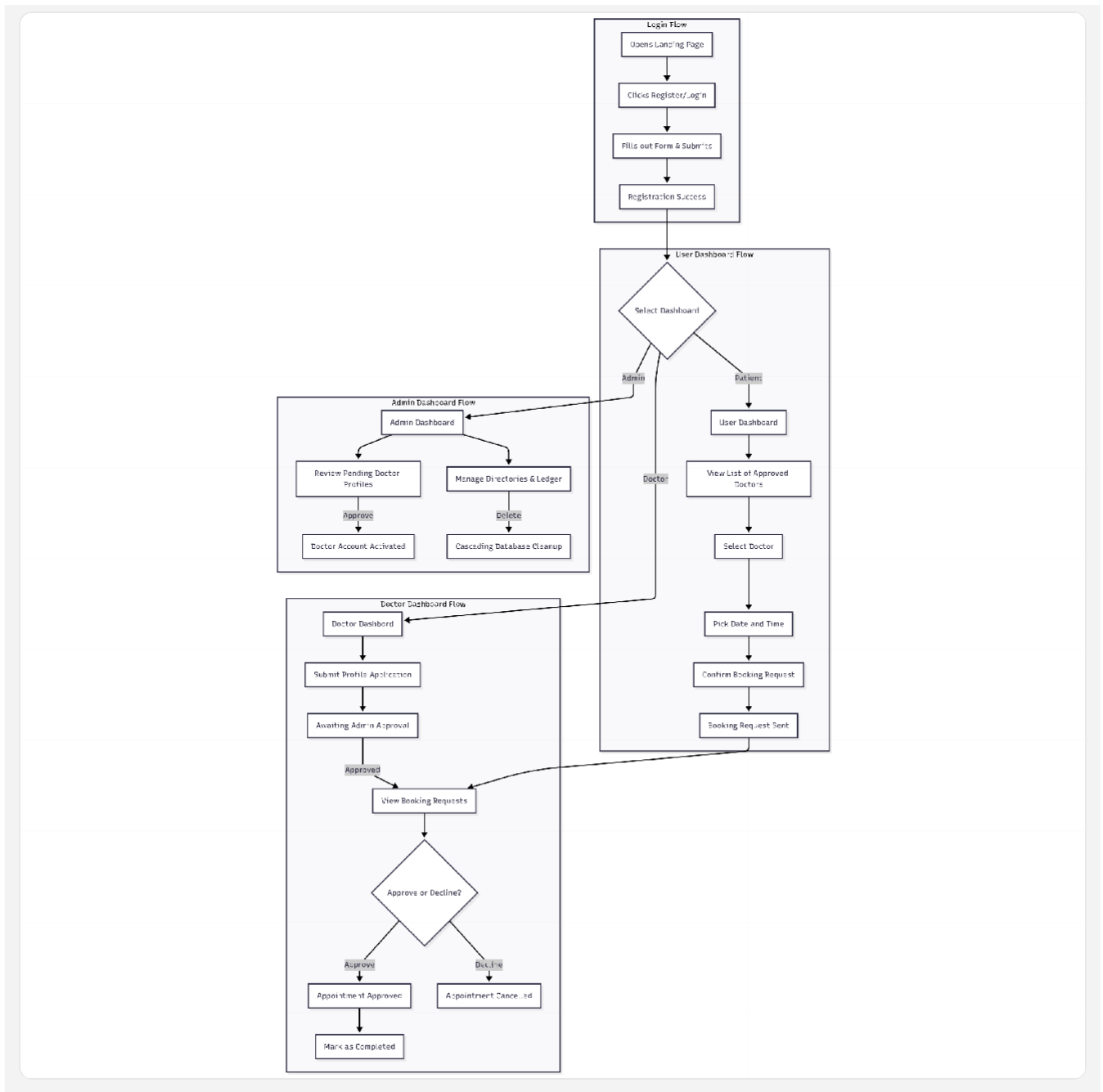
B.1 Horizontal System Flowchart

This flowchart details user paths, from initial registration and landing selection to backend validations and appointment confirmations:



B.2 Vertical System Flowchart

Alternatively, the user verification and scheduling lifecycle can be viewed in a vertical schema structure (detailed below):



Demo link/Drive link : <https://drive.google.com/drive/folders/1-tVVC5F2eUWhXca67CkerzL712TYzVAL?usp=sharing>

GitHub link : <https://github.com/dheereshgoud01/VIP-C2-Addla-Dheeresh-Goud>